

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

Навчально-науковий інститут комп'ютерних систем

Кафедра інженерії програмного забезпечення

Ольга БОСАК

(група АС-226)

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА

**Вебзастосунок для формування користувацького каталогу мистецької
літератури та підтримки читацьких клубів**

Спеціальність:

121 Інженерія програмного забезпечення

Освітньо-професійна програма:

Інженерія програмного забезпечення

Керівник:

Олена АРНАУТОВА

Одеса – 2026

АНОТАЦІЯ

Босак О. С. Вебзастосунок для формування користувацького каталогу мистецької літератури та підтримки читацьких клубів : кваліфікаційна робота бакалавра за спеціальністю «121 Інженерія програмного забезпечення» / Ольга Сергіївна Босак ; керівник Олена Миколаївна Арнаутова. – Одеса : Нац. ун-т «Одес. політехніка», 2026. – 90 с.

Кваліфікаційна робота містить основну текстову частину на 71 сторінках, список використаних джерел з 20 найменувань на 2 сторінках, додаток на 12 сторінках.

У роботі проведено аналіз предметної області та існуючих рішень у сфері популяризації мистецької літератури. Сформульовано функціональні та нефункціональні вимоги згідно зі стандартами розробки програмного забезпечення. Спроектовано модульну трирівневу архітектуру на базі Next.js App Router із використанням Server Actions. Реалізовано каталог мистецької літератури з пошуком та фільтрацією, модуль подання книг з автозаповненням через Google Books API та AI-генерацією описів, інструменти створення читацьких клубів та літературних подій, аналітику читацьких уподобань з експортом звітів. Проведено тестування на трьох рівнях та експлуатаційні випробування, що підтвердили відповідність поставленій меті.

Ключові слова: вебзастосунок, мистецька література, читацький клуб, каталог книг, Next.js, Google Books API, аналітика читання, літературна подія.

ABSTRACT

Bosak O. S. Web application for creating a user catalog of fiction literature and supporting book clubs : bachelor's qualification work in the specialty «121 Software Engineering» / Olha Serhiivna Bosak ; supervisor Olena Mykolaivna Arnautova. – Odesa : Odesa Polytech. Nat. Univ., 2026. – 90 p.

The qualification work contains the main text part on 71 pages, a list of used sources with 20 titles on 2 pages, an appendix on 12 pages.

The work includes an analysis of the subject area and existing solutions in the field of fiction literature promotion. Functional and non-functional requirements were formulated in accordance with software development standards. A modular three-tier architecture based on Next.js App Router with Server Actions. A catalog of fiction literature with search and filtering, a book submission module with auto-fill via Google Books API and AI-generated descriptions, tools for creating book clubs and literary events, and reading preferences analytics with report export have been implemented. Testing at three levels and operational trials were conducted, confirming compliance with the stated objective.

Keywords: web application, fiction literature, book club, book catalog, Next.js, Google Books API, reading analytics, literary event.

ЗМІСТ

Вступ.....	6
1 Специфікація вимог до вебзастосунку.....	7
1.1 Опис предметної області.....	7
1.2 Аналіз аналогів.....	8
1.3 Опис функціональних вимог.....	9
1.4 Опис нефункціональних вимог.....	24
Висновки до розділу 1.....	26
2 Планування процесу розробки програмного продукту.....	27
2.1 Оцінка тривалості розробки.....	27
2.2 Розробка плану виконання.....	32
2.3 Аналіз ризиків.....	33
Висновки до розділу 2.....	35
3 Проєктування вебзастосунку.....	36
3.1 Визначення архітектури застосунку.....	36
3.2 Опис та проєктування програмних класів.....	37
3.3 Структура бази даних.....	40
3.4 Проєктування процесів.....	44
3.5 Проєктування інтерфейсу користувача.....	47
Висновки до розділу 3.....	50
4 Програмна реалізація вебзастосунку.....	51
4.1 Вибір інструментів розробки.....	51
4.2 Технологія створення вебзастосунку.....	52
4.3 Приклади роботи вебзастосунку.....	55
Висновки до розділу 4.....	60
5 Тестування функціональності.....	61
5.1 Запровадження юніт тестування.....	61
5.2 Запровадження інтеграційного тестування.....	62
5.3 Запровадження end-to-end тестування.....	63

	5
Висновки до розділу 5.....	64
6 Експлуатація вебзастосунку.....	65
6.1 Інструкція користувача.....	65
6.2 Опис експериментів та аналіз результатів.....	71
Висновки до розділу 6.....	73
Загальні висновки.....	74
Список використаних джерел.....	75
Додаток А Лістинг програми.....	77

ВСТУП

Актуальність теми роботи. В умовах зниження читацької активності населення існує потреба в сучасних інструментах популяризації мистецької літератури. В Україні відсутня єдина платформа для організації читацьких клубів та літературних подій – існуючі ініціативи мають локальний характер і функціонують у межах окремих спільнот або книжкових магазинів. Це обмежує їх охоплення та ускладнює пошук однодумців. Водночас, наявні сервіси фокусуються переважно на продажі книг, не приділяючи уваги формуванню читацьких спільнот. Розроблюваний вебзастосунок LibProj покликаний масштабувати існуючі читацькі ініціативи, спростити створення нових спільнот та зробити культуру читання доступнішою для аудиторії.

Мета і задачі роботи. Метою роботи є підвищення рівня залученості користувачів до читання мистецької літератури та участі в читацьких клубах шляхом надання інструментів каталогу, спільної взаємодії та аналітики читацьких уподобань. Щоб досягти мети, були поставлені задачі:

- провести аналіз предметної області та існуючих рішень, сформулювати функціональні та нефункціональні вимоги;
- здійснити планування процесу розробки з оцінкою тривалості та аналізом потенційних ризиків;
- спроектувати архітектуру, структуру бази даних та програмні класи;
- виконати програмну реалізацію відповідно до розроблених вимог;
- провести тестування та розгортання готового програмного продукту.

Об’єкт роботи. Процес розробки вебзастосунку для популяризації мистецької літератури та формування читацьких спільнот.

Предмет роботи. Вебзастосунок з інтегрованими інструментами для формування каталогу мистецької літератури, організації читацьких клубів та аналізу читацьких уподобань.

1 СПЕЦИФІКАЦІЯ ВИМОГ ДО ВЕБЗАСТОСУНКУ

1.1 Опис предметної області

Предметна область дослідження охоплює процеси організації читацьких спільнот, розповсюдження літератури та створення середовища для взаємодії між учасниками літературного процесу в цифровому просторі. Вебзастосунок виконує функції для організації подій, формуючи екосистему для популяризації читання.

Основні актори вебзастосунку.

Користувачі (читачі) – фізичні особи, які після реєстрації отримують доступ до повного функціоналу: перегляд та придбання літератури, участь у читацьких активностях, соціальна взаємодія через рецензії та коментарі, можливість продажу книжок іншим користувачам. Користувачі можуть виступати в ролі учасників або організаторів клубів та подій.

Адміністратори – уповноважені особи з розширеними правами доступу для забезпечення належного функціонування: модерація контенту, управління каталогом, обробка заявок на створення спільнот, моніторинг транзакцій, формування аналітичної звітності на основі дій користувачів.

Основними сутностями предметної області є:

- 1) книжки – літературні твори з детальною інформацією (назва, автор, опис, ціна, жанр, рік видання);
- 2) користувачі – профілі з особистою інформацією, історією покупок та активності;
- 3) читацькі клуби – довготривалі об'єднання для регулярного обговорення літератури;
- 4) літературні події – разові або обмежені за кількістю зустрічей заходи;
- 5) замовлення – транзакції покупки книжок через платформу;
- 6) рецензії та оцінки – відгуки користувачів на прочитані книжки;

1.2 Аналіз аналогів

Для визначення переваг розроблюваного вебзастосунку LibProj проведемо аналіз схожих за функціоналом платформ, які орієнтовані на створення читацьких спільнот та взаємодію між читачами.

Goodreads[1] – найбільша у світі соціальна мережа для читачів. Платформа дозволяє створювати віртуальні книжкові полиці, писати рецензії, брати участь у читацьких викликах та дискусійних групах.

LibraryThing[2] – платформа для створення каталогу з особистої бібліотеки. Користувачі можуть додавати книжки, створювати групи за інтересами, організовувати зустрічі. Особливістю є детальна система тегів та можливість імпорту даних з різних джерел.

StoryGraph[3] – платформа, створена як альтернатива Goodreads. Фокусується на персоналізованих рекомендаціях на основі вподобань читача, надає аналітику читацьких звичок, дозволяє створювати читацькі виклики та ділитися враженнями.

Bookclubs[4] – платформа для організації та управління книжковими клубами. Надає інструменти для планування зустрічей, голосування за вибір книжок, ведення дискусій та організації віртуальних зустрічей через відеозв'язок.

Розроблюваний вебзастосунок вирізняється комплексним підходом до організації читацьких подій, поєднуючи в одному вебзастосунку функції комерції, соціальної мережі та організатора подій. Перевагою є можливість самостійного додавання видань до каталогу, що робить платформу більш гнучкою та адаптивною до потреб спільноти. Крім того, LibProj пропонує користувачам доступ до аналітичних даних, що відкриває можливості проведення досліджень читацьких трендів. Іншою перевагою є орієнтація на україномовну аудиторію. Жодна з проаналізованих платформ не враховує специфіку українського книжкового ринку. Це створює унікальну нішу для розроблюваного вебзастосунку, який стане першою комплексною платформою для організації читацьких спільнот, що значно спростить доступ до літературних заходів для україномовних читачів та

сприятиме розвитку читацької культури в Україні. Отже, представимо вищезазначених конкурентів LibProj та основні фактори порівняння для наочного представлення (табл. 1.1).

Таблиця 1.1 – Аналіз конкурентів

Фактор порівняння	LibProj	Goodreads	Library Thing	Story Graph	BookClubs
Організація читацьких клубів та івентів	+	+/-	+/-	-	+
Можливість додавання мистецької літератури	+	-	+	-	-
Рецензії користувачів	+	+	+	+	+/-
Генерація аналітики	+	+	+	-	-
Розраховано на український ринок	+	-	-	-	-

На основі аналізу конкурентів можна зробити висновок, що вебзастосунок LibProj має суттєві переваги порівняно з існуючими рішеннями. Жодна з розглянутих платформ не поєднує в собі повний набір функціоналу, який пропонує розроблюваний вебзастосунок. Найближчими за функціоналом є Goodreads та LibraryThing, проте вони мають обмежені можливості для організації локальних читацьких зустрічей. BookClubs спеціалізується на організації клубів, але не має розвиненої системи каталогізації та аналітики. StoryGraph орієнтований переважно на персональні рекомендації без акценту на спільнотах.

1.3 Опис функціональних вимог

Для опису функціональних вимог представлено діаграму варіантів використання, яка описує роботу вебзастосунку.

Постумови: користувач створив обліковий запис.

Основний успішний сценарій (або основний процес):

1. Користувач вводить ім'я, пошту, пароль, підтвердження паролю.
2. Система перевіряє коректність введених даних.
3. Дані введені коректно, тому система активує обліковий запис і перенаправляє на сторінку входу.

Альтернативні сценарії:

- 3а. Дані введені некоректно, введена пошта вже використовується. Система повідомляє про помилку та пропонує ввести пошту повторно. Повернення до кроку 1.
 - 3б. Не всі поля заповнені. Система відображає повідомлення про незаповнені поля. Повернення до кроку 1.
-

Наступним розглянемо сценарій авторизації (див. сценарій 1.2).

Сценарій 1.2 – Авторизація

Актор: зареєстрований користувач.

Мета: користувач хоче увійти в систему.

Передумови: користувач зареєстрований у системі, має доступ до мережі інтернет та знаходиться на сторінці входу.

Постумови: користувач успішно отримав доступ до свого облікового запису.

Основний успішний сценарій (або основний процес):

1. Користувач вводить пошту та пароль на сторінці входу.
2. Система перевіряє коректність введених даних.
3. Дані введені коректно, тому система входить та відкриває головну сторінку.

Альтернативні сценарії:

- 3а. Дані введені некоректно, користувач ввів неіснуючу пошту. Система повідомляє про відсутність облікового запису та пропонує зареєструватися. Повернення до кроку 1.

3б. Дані введені некоректно, система повідомляє клієнта про помилку та пропонує повторити введення. Повернення до кроку 1.

Наступний сценарій демонструє процес пошуку книги. Користувач може використовувати пошукові запити або фільтри (див. сценарій 1.3).

Сценарій 1.3 – Перегляд та пошук літератури

Актор: користувач.

Мета: користувач хоче знайти та переглянути наявну літературу.

Передумови: користувач має доступ до мережі інтернет, знаходиться на головній сторінці системи, та у системі є каталог книг.

Постумови: користувач переглянув доступні книги та може додати книгу в кошик або переглянути її детальну інформацію.

Основний успішний сценарій (або основний процес):

1. Користувач обирає категорію книг або вводить ключові слова в пошук.
2. Система відображає список відповідних книг.
3. Користувач переглядає список, використовуючи фільтри (жанр, автор, ціна).
4. Користувач обирає книгу для перегляду детальної інформації.
5. Система відображає сторінку з детальним описом книги, ціною та відгуками.

Альтернативні сценарії:

- 2а. За введеним пошуковим запитом нічого не знайдено. Система повідомляє, що немає результатів за введеним запитом. Повернення до кроку 1.
-

У наступному сценарії описано, як користувач може додати обрану літературу до кошика. Це дозволяє зберегти товари для подальшого оформлення замовлення, порівняти обрані позиції та прийняти остаточне рішення щодо покупки літератури. Користувач має можливість змінювати кількість обраних одиниць товару або видаляти їх із кошика у будь-який момент (див. сценарій 1.4)

Сценарій 1.4 – Додавання в кошик

Актор: користувач.

Мета: користувач хоче додати книгу до кошика для подальшого оформлення замовлення.

Передумови: користувач переглядає книгу в каталозі.

Постумови: користувач успішно додав книгу в кошик та може переглянути кошик.

Основний успішний сценарій (або основний процес):

1. Користувач відкриває сторінку книги.
2. Користувач запитує додавання книги до кошика.
3. Система додає книгу в кошик та показує підтвердження.

Альтернативні сценарії:

2а. Обрана книга недоступна для покупки. Система відображає повідомлення про недоступність та не дозволяє користувачу додати літературу в кошик.

Далі маємо сценарій, який описує повний процес оформлення покупки: від перегляду кошика до підтвердження оплати (див. сценарій 1.5).

Сценарій 1.5 – Оформлення замовлення

Актор: зареєстрований користувач.

Мета: користувач хоче оформити замовлення та оплатити його.

Передумови: користувач зареєстрований у системі та додав товари в кошик.

Постумови: замовлення оформлене та користувач отримав підтвердження про успішну оплату.

Основний успішний сценарій (або основний процес):

1. Користувач відкриває кошик.
2. Система відображає список товарів у кошику та загальну вартість.
3. Користувач ініціює оформлення замовлення.
4. Система відображає форму введення даних та вибору способу оплати.

5. Користувач вводить дані та підтверджує оплату.
6. Система передає дані в платіжну систему.
7. Платіжна система підтверджує успішну оплату.
8. Система створює замовлення та відправляє клієнту підтвердження.

Альтернативні сценарії:

- 2а. Кошик пустий. Система повідомляє клієнта та пропонує додати товари до кошику.
 - 7а. Оплата не пройшла. Система повідомляє клієнта про невдалу транзакцію та пропонує повторити спробу.
-

Наступний сценарій демонструє, як користувач може створити власну літературну подію. Він заповнює всю необхідну інформацію про захід, після чого вона зберігається в базі даних для модерації адміністратором (див. сценарій 1.6).

Сценарій 1.6 – Планування літературної події

Актор: зареєстрований користувач.

Мета: користувач хоче створити нову літературну подію.

Передумови: користувач авторизований у системі.

Постумови: подія створена та очікує на модерацію.

Основний успішний сценарій (або основний процес):

1. Користувач переходить до розділу літературних подій
2. Користувач ініціює створення нової події.
3. Система відображає форму для заповнення (назва події, опис, дата, час, місце/формат, контактні дані, кількість зустрічей, максимальна кількість учасників, книги для обговорення).
4. Користувач заповнює форму та підтверджує збереження.
5. Система створює подію зі статусом «Очікує модерації» та повідомляє користувача.

Альтернативні сценарії:

- 5а. Користувач не заповнив обов'язкові поля. Система відображає повідомлення про незаповнені поля. Повернення до кроку 4.
- 5б. Вказана дата події є минулою. Система відображає повідомлення про невірну дату. Повернення до кроку 4.
-

Наступний сценарій схожий на попередній, так як користувач також може створити власний читацький клуб. Він заповнює всю інформацію, після чого вона зберігається в базі даних для модерації адміністратором (див. сценарій 1.7).

Сценарій 1.7 – Планування читацького клубу

Актор: зареєстрований користувач.

Мета: користувач хоче створити новий читацький клуб.

Передумови: користувач авторизований у системі.

Постумови: клуб створений та очікує на модерацію.

Основний успішний сценарій (або основний процес):

1. Користувач переходить до розділу читацьких клубів.
2. Користувач ініціює створення нового клубу.
3. Система відображає форму для заповнення (назва, мета, опис, формат проведення, максимальна кількість учасників, книги для обговорення).
4. Користувач заповнює форму та підтверджує збереження.
5. Система створює клуб зі статусом «Очікує модерації».

Альтернативні сценарії:

- 5а. Користувач не заповнив обов'язкові поля. Система відображає повідомлення про незаповнені поля. Повернення до кроку 4.
- 5б. Не вибрано жодної книги для обговорення. Система повідомляє про помилку та пропонує обрати хоча б одну книгу. Повернення до кроку 4.
-

У наведеному сценарії користувач має змогу опублікувати книжку. Він надсилає публікацію на модерацію (див. сценарій 1.8).

Сценарій 1.8 – Додавання книжки (користувач)

Актор: зареєстрований користувач.

Мета: користувач хоче додати нову книгу до каталогу платформи.

Передумови: користувач авторизований у системі.

Постумови: книга додана до системи зі статусом «Очікує модерації».

Основний успішний сценарій (або основний процес):

1. Користувач переходить до розділу додавання літератури.
2. Користувач ініціює додавання нової книги.
3. Система відображає форму для введення даних (назва, автор, жанр, ціна, рік видання, ISBN, опис, мова, видавництво, кількість сторінок).
4. Користувач заповнює форму та завантажує зображення обкладинки.
5. Користувач відправляє книгу на модерацію.
6. Система зберігає книгу зі статусом «Очікує модерації».

Альтернативні сценарії:

- 3а. Користувач вводить ISBN. Система автоматично заповнює частину полів на основі введенного ISBN.
 - 5а. Користувач не заповнив обов'язкові поля. Система відображає повідомлення про помилку. Повернення до кроку 4.
 - 5б. Система виявляє схожу книгу в каталозі та повідомляє про можливий дублікат. пропонує переглянути існуючу або продовжити додавання.
 - 6а. Розмір зображення перевищує встановлений ліміт по розміру файлу. Система повідомляє про необхідність завантажити менше зображення. Повернення до кроку 4.
-

Нижче наведений сценарій описує процес додавання нової книги адміністратором. Після заповнення всіх необхідних полів книга з'являється в каталозі та стає доступною для перегляду й замовлення користувачами. Адміністратор вносить основну інформацію про додану літературу (див. сценарій 1.9).

Сценарій 1.9 – Додавання книжки (адміністратор)

Актор: адміністратор.

Мета: адміністратор хоче додати нову книгу до каталогу.

Передумови: каталог книг доступний користувач авторизований як адміністратор.

Постумови: нова книга додана до каталогу.

Основний успішний сценарій (або основний процес):

1. Адміністратор відкриває панель керування.
2. Адміністратор обирає дію додавання літератури.
3. Система відображає форму для введення даних (назва, slug, автор, опис, ціна, категорія, зображення, кількість сторінок, мова, ISBN, банер).
4. Адміністратор заповнює форму та підтверджує додавання.
5. Система додає товар у каталог.

Альтернативні сценарії:

- 5а. Адміністратор не заповнив усі обов'язкові поля. Система відображає повідомлення про помилку. Повернення до кроку 4.
-

Даний сценарій дозволяє користувачу або адміністратору автоматично згенерувати короткий опис книги на основі введених даних. (див. сценарій 1.10).

Сценарій 1.10 – Генерація опису книги

Актор: користувач або адміністратор (при додаванні книги).

Мета: автоматично згенерувати короткий опис книги.

Передумови: користувач знаходиться у формі додавання/редагування книги. AI-сервіс доступний та налаштований, введено мінімально необхідні дані про книгу (назва, автор, жанр).

Постумови: згенерований опис відображається у відповідному полі форми та готовий до редагування або збереження.

Основний успішний сценарій (або основний процес):

1. Користувач відкриває форму додавання книги.

2. Користувач завантажує файл або вводить базову інформацію (назва, жанр, автор).
3. Користувач запитує генерацію опису засобами AI.
4. Система надсилає дані до AI-сервісу.
5. AI сервіс формує текст опису.
6. Застосування підставляє згенерований текст у відповідне поле.

Альтернативні сценарії:

- 5a. AI сервіс не може згенерувати опис. Система повідомляє про помилку та пропонує ввести опис вручну.
-

Наступний сценарій дозволяє зареєстрованому користувачу залишити рецензію на книгу (див. сценарій 1.11).

Сценарій 1.11 – Залишення рецензії

Актор: зареєстрований користувач.

Мета: додати рецензію на книгу та взяти участь в обговоренні.

Передумови: користувач авторизований у системі, книга існує в каталозі та доступна для рецензування, користувач ще не залишав рецензію на цю книгу.

Постумови: рецензія збережена в базі даних, оновлено загальний рейтинг книги та кількість рецензій, рецензія відображається на сторінці книги. Якщо кількість рецензій досягла 10, згенеровано AI-резюме відгуків.

Основний успішний сценарій (або основний процес):

1. Користувач відкриває сторінку книги.
2. Користувач ініціює додавання рецензії.
3. Система відображає форму: текст рецензії, рейтинг.
4. Користувач заповнює форму та надсилає рецензію.
5. Система зберігає рецензію та оновлює рейтинг книги.
6. Рецензія відображається серед інших на сторінці книги.
7. Система перевіряє кількість рецензій: якщо їх 10 або більше, генерується AI-резюме з визначенням тональності, позитивних та негативних аспектів.

8. Користувач обирає рецензію іншого користувача та ініціює відповідь.
9. Система відображає форму відповіді.
10. Користувач вводить текст та надсилає відповідь.
11. Відповідь відображається під рецензією. Інші користувачі також можуть відповідати на інші відгуки, формуючи дерево обговорення.

Альтернативні сценарії:

- 4а. Не заповнені обов'язкові поля. Система повідомляє про помилку, повернення до кроку 3.
 - 4б. Користувач намагається додати повторну рецензію. Система повідомляє про це та пропонує відредагувати існуючу.
 - 7а. Кількість рецензій менша за 10. Система не генерує AI-резюме, сценарій продовжується з кроку 8.
-

Переходимо до сценарію 1.12, який дозволяє зареєстрованому користувачу приєднатись до однієї із існуючих літературних подій.

Сценарій 1.12 – Приєднання до літературної події/клубу

Актор: зареєстрований користувач.

Мета: увійти до команди певної літературної події/клубу.

Передумови: користувач авторизований у системі, подія/клуб існує, має статус «активна» та не досягла ліміту учасників та дата початку події ще не настала.

Постумови: користувач доданий до списку учасників події/клубу, оновлено кількість зареєстрованих учасників та користувач отримав доступ до внутрішньої сторінки події.

Основний успішний сценарій (або основний процес):

1. Користувач переходить до списку усіх літературних подій/клубів.
2. Користувач обирає цікаву для себе подію/клуб та запитує про приєднання.
3. Система додає користувача до списку учасників події/клубу.
4. Користувач отримує доступ до внутрішньої сторінки.

Альтернативні сценарії:

- 3а. Клуб/подія заповнені (ліміт учасників досягнуто). Система повідомляє користувача про відсутність вільних місць. Повернення до кроку 1.
- 3б. Користувач вже є учасником клубу/події. Система повідомляє про це та пропонує переглянути інші клуби або події. Повернення до кроку 1.
-

Адміністратор може оновити інформацію про книгу в каталозі. Це може редагування або видалення з каталогу (див. сценарій 1.13).

Сценарій 1.13 – Управління існуючим каталогом

Актор: адміністратор.

Мета: перегляд, редагування або видалення книг у каталозі.

Передумови: адміністратор має доступ до панелі керування, у системі є щонайменше одна книга.

Постумови: адміністратор виконав необхідні дії з книгами в каталозі.

Основний успішний сценарій (або основний процес):

1. Адміністратор відкриває панель керування.
2. Адміністратор переходить до розділу каталогу книг.
3. Система відображає список книг з можливістю пошуку та фільтрації.
4. Адміністратор обирає книгу для перегляду.
5. Система відображає детальну інформацію про книгу та кнопки дій.
6. Адміністратор обирає одну з дій: редагувати, видалити, змінити статус.
7. Система виконує обрану дію та оновлює каталог.

Альтернативні сценарії:

- 3а. У каталозі відсутні книги. Система відображає повідомлення та пропонує додати першу книгу.
- 6а. Адміністратор залишив порожніми обов'язкові поля. Система відображає помилку валідації. Повернення до кроку 6.
- 6б. Книга вже замовлена. Система попереджає, що книга має активні замовлення та запитує користувача про підтвердження дії.
-

Наступним маємо сценарій, який демонструє як адміністратор переглядає, оновлює статус або скасовує замовлення користувачів. Це дозволяє підтримувати актуальність інформації в системі (див. сценарій 1.14).

Сценарій 1.14 – Управління існуючими замовленнями

Актор: адміністратор.

Мета: перегляд, оновлення статусу або скасування наявних замовлень користувачів.

Передумови: є щонайменше одне оформлене замовлення та адміністратор має доступ до панелі керування.

Постумови: адміністратор переглянув замовлення, оновив його статус або скасував при потребі.

Основний успішний сценарій (або основний процес):

1. Адміністратор відкриває панель керування.
2. Переходить до розділу замовлень.
3. Система відображає список існуючих замовлень (з номером, клієнтом, датою).
4. Адміністратор обирає конкретне замовлення для перегляду.
5. Система відображає деталі замовлення (товари, клієнт, оплата, доставка).
6. Адміністратор обирає одну з дій: (змінити статус, скасувати замовлення)
7. Система оновлює інформацію та підтверджує успішне збереження змін.

Альтернативні сценарії:

- 5а. Замовлення за вказаними параметрами не знайдено. Система відображає повідомлення про відсутність результатів. Повернення до кроку 3.
-

Наступний сценарій дозволяє адміністратору модерувати запити на створення нових подій та літературних клубів. Адміністратор переглядає подані заявки, оцінює їх відповідність правилам та приймає рішення про схвалення або відхилення запиту. У разі відхилення вебзастосунок надсилає користувачу повідомлення із зазначенням причини, що дає можливість внести корективи та подати заявку повторно (див. сценарій 1.15).

Сценарій 1.15 – Модерація подій та клубів

Актор: адміністратор.

Мета: перевірити та затвердити/відхилити створені події або клуби.

Передумови: адміністратор авторизований у системі.

Постумови: подія/клуб опублікована або відхилена.

Основний успішний сценарій (або основний процес):

1. Адміністратор переходить до розділу панелі керування.
2. Система відображає список подій/клубів зі статусом «Очікує модерації».
3. Адміністратор вибирає подію/клуб для перегляду.
4. Система відображає детальну інформацію.
5. Адміністратор приймає рішення: затверджує або відхиляє подію/клуб.
6. Система змінює статус на «Опубліковано» або «Відхилено».

Альтернативні сценарії:

- 5а. Адміністратор відхиляє подію/клуб. Система вимагає обов'язково вказати причину (дублікат, невідповідність правилам). Повернення до кроку 5.
-

Далі розглянемо сценарій, коли адміністратор перевіряє додану користувачами літературу та приймає рішення публікацію (див. сценарій 1.16).

Сценарій 1.16 – Модерація доданих книжок

Актор: адміністратор.

Мета: перевірити, схвалити або відхилити книжку, додану користувачем.

Передумови: існують неопрацьовані книжки та адміністратор авторизований.

Постумови: книжка змінює статус: схвалена та додана до каталогу або відхилена.

Основний успішний сценарій (або основний процес):

1. Адміністратор відкриває розділ модерації книжок.
2. Система показує список книжок, які очікують модерації.
3. Адміністратор обирає книжку для перегляду.

4. Система відкриває детальну інформацію (назва, автор, жанр, опис, рік видання, ISBN (за необхідності), обкладинка, користувач що додав).
5. Адміністратор перевіряє коректність даних та наявність дублікатів.
6. Адміністратор приймає рішення: схвалює або відхиляє книжку.
7. Система оновлює статус книжки та надсилає повідомлення користувачу.

Альтернативні сценарії:

- 4а. Система виявляє схожу книжку в каталозі та повідомляє адміністратора про можливий дублікат.
 - 5а. Адміністратор відхиляє книжку. Система вимагає обов'язково вказати причину відхилення. Повернення до кроку 6.
 - 6б. Адміністратор редагує інформацію про книжку перед схваленням. Система зберігає зміни.
-

Наступний сценарій використання – адміністратор генерує аналітику за певний період на основі даних вебзастосунку (див. сценарій 1.17).

Сценарій 1.17 – Генерація аналітики (адміністратор)

Актор: адміністратор.

Мета: згенерувати детальну аналітику.

Передумови: адміністратор авторизований у системі.

Постумови: аналітичний звіт згенерований та доступний для завантаження.

Основний успішний сценарій (або основний процес):

1. Адміністратор відкриває панель аналітики.
2. Адміністратор обирає тип аналітики (активність користувачів, популярність літератури, події/клуби).
3. Адміністратор запускає генерацію аналітики.
4. Система обробляє дані та формує аналітику.
5. Адміністратор завантажує аналітику у обраному форматі (PDF, Excel, CSV).

Альтернативні сценарії:

- 2а. Не вказано обов'язкові параметри. Система відображає повідомлення про помилку валідації. Повернення до кроку 2.
 - 4а. Обсяг даних для обробки є великим. Система відображає індикатор прогресу.
 - 5а. Під час генерації сталась помилка. Система пропонує змінити параметри. Повернення до кроку 2.
-

Розглянемо наступний сценарій використання – перегляд аналітики за певний період на основі даних користувача (див. сценарій 1.18).

Сценарій 1.18 – Перегляд аналітики (користувач)

Актор: зареєстрований користувач.

Мета: переглянути публічну аналітику читацьких трендів.

Передумови: користувач авторизований у системі.

Постумови: користувач ознайомився з аналітичними даними.

Основний успішний сценарій (або основний процес):

1. Користувач переходить до розділу аналітики.
2. Система відображає доступні розділи (популярні книги, тренди жанрів, активність клубів).
3. Користувач обирає цікавий розділ.
4. Система показує візуалізовані дані (графіки, діаграми, топ-списки).
5. Користувач може фільтрувати дані за періодом та категоріями.
6. Система оновлює візуалізацію відповідно до фільтрів.

Альтернативні сценарії:

- 4а. Даних для відображення недостатньо. Система показує мінімальну доступну статистику.
 - 6а. Обраний період не містить даних. Система повідомляє про відсутність даних та пропонує змінити фільтри. Повернення до кроку 5.
-

1.4 Опис нефункціональних вимог

Надійність:

- 1) вебзастосунок повинен забезпечувати доступність основних функцій не менше 95% часу протягом місяця;
- 2) резервні копії бази даних мають створюватися автоматично щоденно, з можливістю відновлення даних за останні 7 днів;
- 3) при збої окремих компонентів вебзастосунок повинен продовжувати роботу, повідомляючи користувачів про тимчасову недоступність функцій.

Продуктивність:

- 1) головна сторінка та каталог книг повинні завантажуватися не довше 3 секунд при швидкості інтернету від 10 Мбіт/с;
- 2) результати пошуку мають відображатися протягом 2 секунд для запитів у межах 1000 результатів;
- 3) вебзастосунок повинен підтримувати одночасну роботу щонайменше 100 активних користувачів без помітного зниження швидкодії;
- 4) API запити повинні використовувати пагінацію та кешування для зменшення навантаження на сервер, з часом відповіді не більше 500 мс для стандартних операцій.

Безпека:

- 1) всі форми введення даних повинні мати валідацію на стороні клієнта та сервера для запобігання SQL-ін'єкцій та XSS-атак;
- 2) сесії користувачів повинні автоматично завершуватися після 30 хвилин неактивності;
- 3) платіжні дані користувачів не повинні зберігатися в системі – всі транзакції обробляються через захищений API PayPal.

Супроводжуваність:

- 1) архітектура вебзастосунку повинна базуватися на модульному принципі з чітким розподілом на компоненти UI, API routes, сервіси та утиліти;

- 2) код повинен використовувати TypeScript для статичної типізації та містити JSDoc коментарі для складних функцій і компонентів;
- 3) база даних повинна використовувати Prisma ORM з міграціями для контролю версій схеми даних та забезпечення типобезпеки на рівні запитів;
- 4) код повинен покриватися unit та інтеграційними тестами з мінімальним покриттям 70% для критичних модулів, з Jest та React Testing Library.

Зручність використання:

- 1) реєстрація нового користувача не повинна займати більше 2 хвилин та вимагати заповнення не більше 5 обов'язкових полів;
- 2) всі форми повинні мати зрозумілі повідомлення про помилки з вказівками щодо їх виправлення;
- 3) критичні дії повинні вимагати підтвердження від користувача;
- 4) мобільна версія вебзастосунку повинна забезпечувати доступ до всіх основних функцій без горизонтальної прокрутки на екранах від 320px.

Висновки до розділу 1

У даному розділі було проведено аналіз вимог до вебзастосунку LibProj. В результаті опису предметної області визначено основних акторів, ключові бізнес-процеси та сутності, що дозволило окреслити межі та взаємозв'язки. Аналіз аналогів виявив відсутність українських платформ для читацьких спільнот. Це підтверджує унікальність платформи, що поєднує можливості каталогу літератури, організації читацьких клубів та аналітичних інструментів. Функціональні вимоги систематизовано у вигляді 18 сценаріїв використання. Кожен сценарій містить передумови, основний потік та альтернативні шляхи виконання, що забезпечує повноту опису функціоналу. Нефункціональні вимоги структуровано категоріями: надійність, продуктивність, безпека, супроводжуваність та зручність використання.

2 ПЛАНУВАННЯ ПРОЦЕСУ РОЗРОБКИ ПРОГРАМНОГО ПРОДУКТУ

2.1 Оцінка тривалості розробки

Для оцінювання розміру та тривалості розробки програмного забезпечення обрано метод Use Case Points[5] – (UCP), запропонований Густавсоном та Карлсоном, де ключову роль відіграють варіанти використання. На цьому етапі розробки доступні саме функціональні вимоги, а не точні технічні специфікації, тому інші методи були б менш точними. Виконаємо оцінку акторів (UAW):

- зареєстрований користувач – складний – 3;
- новий користувач – складний – 3;
- адміністратор – складний – 3;
- платіжна система – простий (API) – 1, тоді маємо:

$$UAW = 3 + 3 + 3 + 1 = 10.$$

Перейдемо до оцінки варіантів використання (UUCW) за кількістю транзакцій. Сценарії з 1–3 транзакціями класифікуються як прості (вага 5), з 4–7 – як середні (вага 10), з 8 і більше – як складні (вага 15). У стовпці «Кількість транзакцій» наведено послідовність транзакцій кожного сценарію (табл. 2.1).

Таблиця 2.1 – Оцінка варіантів використання за транзакціями

Сценарій	Тип	Вага	Кількість транзакцій
1	2	3	4
Реєстрація	Простий	5	Введення даних, валідація, автоматичний вхід
Авторизація	Простий	5	Введення даних, перевірка, вхід
Перегляд та пошук літератури	Середній	10	Введення запиту, запит до БД, застосування фільтрів, відображення результатів

Продовження таблиці 2.1

1	2	3	4
Додавання в кошик	Простий	5	Відкриття сторінки, натискання кнопки, збереження
Оформлення замовлення	Середній	10	Перегляд кошика, введення адреси доставки, вибір методу оплати, підтвердження замовлення, оплата, оновлення статусу
Планування літературної події	Середній	10	Відкриття розділу, заповнення форми, перевірка дублікатів та книг, збереження запиту
Планування читацького клубу	Середній	10	Відкриття розділу, заповнення форми, перевірка дублікатів та книг, збереження запиту
Додавання книжки (користувач)	Середній	10	Відкриття розділу, пошук, заповнення форми, генерація опису, перевірка дублікатів, збереження
Генерація опису книги	Середній	10	Введення даних у форму, відправка запиту, перевірка сесії, валідація параметрів, генерація
Залишення рецензії	Середній	10	Відкриття сторінки, заповнення форми, надсилання, збереження, перерахунок середнього рейтингу
Приєднання до події/клубу	Простий	5	Перехід до списку та вибір, перевірка місткості та реєстрація, додавання до учасників та email-підтвердження
Перегляд аналітики (користувач)	Середній	10	Відкриття розділу, завантаження даних, відображення дашборду, вибір фільтру/періоду, оновлення даних, перевірка та нарахування досягнень
Додавання літератури (адміністратор)	Середній	10	Відкриття адмін-панелі, вибір дії, відображення форми, заповнення даних, збереження

Кінець таблиці 2.1

1	2	3	4
Управління існуючим каталогом	Середній	10	Відкриття адмін-панелі, перехід до розділу, завантаження списку книг, вибір книги, відображення деталей, вибір дії
Управління існуючими замовленнями	Середній	10	Відкриття адмін-панелі, перехід до розділу, завантаження списку, вибір замовлення, відображення деталей, вибір дії, оновлення статусу
Модерація подій та клубів	Середній	10	Відкриття розділу, завантаження списку запитів, вибір запиту, відображення деталей, прийняття рішення, оновлення статусу та створення сутності
Модерація доданої літератури	Середній	10	Завантаження списку заявок з метриками, вибір книжки, відображення деталей, перевірка даних, прийняття рішення, оновлення статусу
Генерація аналітики (адміністратор)	Середній	10	Відкриття адмін-панелі, вибір типу та часового діапазону, запуск генерації, паралельна обробка даних, збереження звіту, завантаження результату

Користуючись даними наведеними в табл. 2.1 можемо визначити кількість кожного із типів сценаріїв: простих – 4; $4 \times 5 = 20$, середніх – 14; $14 \times 10 = 140$, складних – $0 \times 15 = 0$. Тоді сума всіх варіантів використання:

$$UUCW = 20 + 140 + 0 = 160.$$

Тепер можемо визначити показник UCP формула (2.1) (Враховуємо, що %REUSE = 0, бо маємо першу ітерацію):

$$UCP = UAW + UUCW. \quad (2.1)$$

Тоді в результаті отримаємо:

$$UCP = 10 + 160 = 170.$$

Далі перейдемо до етапу 3 – оцінки технічних факторів (табл. 2.2).

Таблиця 2.2 – Оцінка технічних факторів

Фактор	Вага	Оцінка	Добуток	Пояснення
T1 Розподіленість системи	2	2	4	Клієнт-серверна архітектура
T2 Час відгуку	1	3	3	Швидке завантаження каталогу, обробка платежів
T3 Ефективність	1	4	4	Інтуїтивний інтерфейс
T4 Складність обробки	1	3	3	AI-генерація описів, складна аналітика
T5 Повторне використання коду	1	2	2	Модульна архітектура, компоненти UI
T6 Простота інсталяції	0,5	5	2,5	Вебзастосування, інсталяція відсутня
T7 Простота використання	0,5	4	2	Зрозумілий UI/UX для всіх категорій користувачів
T8 Портативність	2	3	8	Вебзастосунок, працює в усіх браузерах і пристроях
T9 Простота змін	1	3	3	Модульна структура
T10 Паралельні обчислення	1	2	2	Обробка множинних користувачів та замовлень
T11 Засоби захисту	1	4	4	Захист платіжних даних, авторизація, модерація
T12 Доступ до третьої сторони	1	2	2	Інтеграція з платіжними системами
T13 Спеціальне навчання	1	2	2	Інтуїтивний інтерфейс, мінімальне навчання

Визначимо суму добутків вагових коефіцієнтів і оцінок для кожного з технічних факторів:

$$TFactor = 4 + 3 + 4 + 3 + 2 + 2,5 + 2 + 8 + 3 + 2 + 4 + 2 + 2 = 41,5.$$

Наступною обчислимо оцінку технічного фактору формула (2.2):

$$TCF = 0,6 + (0,01 \times TFactor). \quad (2.2)$$

Тоді в результаті отримаємо:

$$TCF = 0,6 + 0,01 \times 41,5 = 0,6 + 0,415 = 1,015.$$

Далі перейдемо до наступного етапу 4 – оцінки зовнішніх факторів, яка дає коефіцієнт для організаційних ризиків при розробці (табл. 2.3).

Таблиця 2.3 – Оцінка зовнішніх факторів

Фактор	Вага	Оцінка	Добуток	Пояснення
F1 Знайомство з процесом розробки	1,5	5	7,5	Ознайомленість з предметною областю та веб-розробкою
F2 Досвід подібних проєктів	0,5	5	2,5	Є досвід створення схожих застосувань
F3 Досвід ООП	1	5	5	Володіння ООП підходом
F4 Досвідченість аналітика	0,5	4	2	Достатньо досвіду для збору вимог
F5 Мотивація	1	5	5	Висока мотивація
F6 Стабільність вимог	2	5	10	Вимоги середньо стабільні, можливі зміни
F7 Зайнятість	-1	0	0	Повна залученість
F8 Складність мови	-1	2	-2	Мова середньої складності

Можемо визначити значення EFactor шляхом додавання добутків вагових коефіцієнтів та оцінок для кожного з зовнішніх факторів:

$$\text{EFactor} = 7,5 + 2,5 + 5 + 2 + 5 + 10 + 0 - 2 = 30.$$

Тепер знайдемо оцінку зовнішнього фактору за формулою (2.3):

$$\text{EF} = 1,4 + (-0,03 \times \text{EFactor}). \quad (2.3)$$

Тоді в результаті отримаємо:

$$\text{EF} = 1,4 + (-0,03 \times 30) = 1,4 - 0,9 = 0,5.$$

Перейдемо до останнього етапу результуючих оцінок формула (2.4):

$$\text{AUCP} = \text{UCP} \times \text{TCF} \times \text{EF}. \quad (2.4)$$

Тоді в результаті отримаємо:

$$\text{AUCP} = 170 \times 1,015 \times 0,5 \approx 86,28.$$

Згідно з методологією, далі необхідно проаналізувати фактори оточення. Підрахунок негативних факторів: F1-F6 з оцінкою < 3 : немає (всі ≥ 3), F7-F8 з оцінкою > 3 : немає. Загальна кількість негативних факторів = 0. Оскільки кількість негативних факторів = 2 (≤ 2), використовуємо 20 людино-годин на UCP. Трудомісткість = $\text{AUCP} \times 20 = 86,26 \times 20 = 1725,5$ людино-годин. Тобто тривалість виконання буде становити $1725,5 / 40 = 43,1$ тижні.

2.2 Розробка плану виконання

На основі структури процесу розробки програмного забезпечення було сформовано WBS, що складається з семи основних етапів:

- 1) специфікація вимог до вебзастосунку (01.09.2025 – 24.10.2025);
- 2) планування процесу розробки (20.10.2025 – 28.11.2025);
- 3) проєктування вебзастосунку (24.11.2025 – 06.02.2026);
- 4) програмна реалізація вебзастосунку (01.12.2025 – 13.03.2026);
- 5) тестування функціональності (19.01.2026 – 20.03.2026);
- 6) експлуатація вебзастосунку (16.03.2026 – 17.04.2026);
- 7) підготовка звіту (09.03.2026 – 30.04.2026).

Загальна оцінка трудомісткості становить 1725,5 робочі години. При тривалості робочого дня 8 годин сумарна тривалість проєкту становить 221,2 робочих днів. Завдяки паралельному виконанню етапів, тривалість – 170 робочих днів. Отже, побудуємо діаграму Ганта, яка зображена на рис. 2.1.

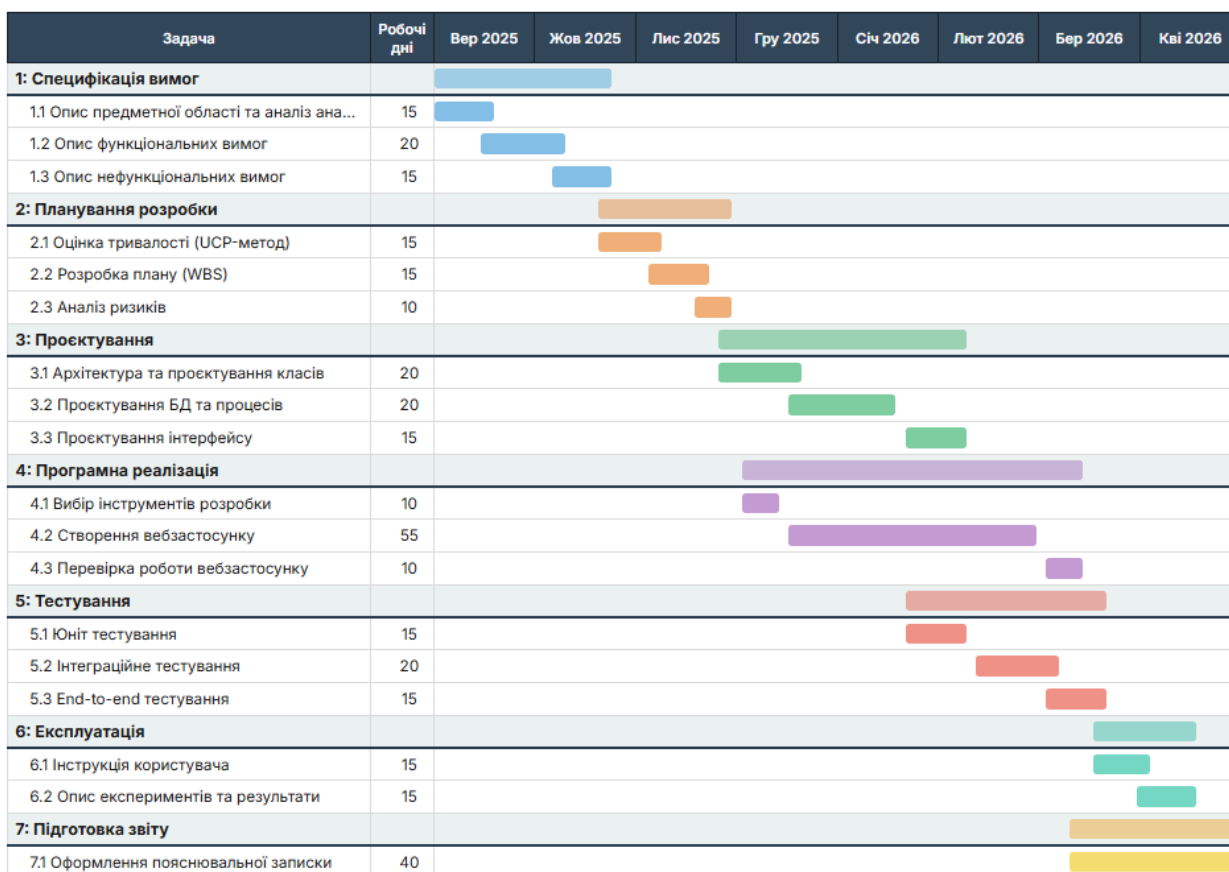


Рисунок 2.1 – Діаграма Ганта

Дана діаграма забезпечує візуалізацію послідовності виконання задач, їх взаємозв'язків, що дає можливість ефективно контролювати хід реалізації.

2.3 Аналіз ризиків

У реалізації важливо завчасно визначити потенційні ризики, що можуть вплинути на строки, якість та обсяг виконання робіт. Наведемо основні ризики, оцінку їх імовірності, масштабу впливу та стратегію реагування (табл. 2.4).

Таблиця 2.4 – Оцінка ризиків

№	Ризик	Імовірність	Вплив	Реагування
R1	Перевищення запланованих термінів виконання	Висока	Високий	Зниження (буфер часу, деталізація плану)
R2	Недостатній досвід роботи з окремими технологіями	Висока	Високий	Зниження(навчання, вибір стабільних технологій)
R3	Перерви через непередбачувані обставини	Середня	Середній	Прийняття + буфер часу
R4	Емоційне виснаження або втрата мотивації	Середня	Середній	Зниження (регулярні перерви)
R5	Помилки в реалізації, що потребують переробок	Висока	Середній	Зниження (Тестування)
R6	Недостатній час на тестування	Середня	Високий	Зниження (виділення тестового етапу)
R7	Втрата або пошкодження даних	Середня	Критичний	Ухилення/зниження (Git)
R8	Труднощі інтеграції зовнішніх сервісів	Низька	Середній	Зниження (використання SDK, документація)
R9	Неточності у вимогах	Низька	Середній	Ухилення (рання конкрет. вимог)
R10	Технічні проблеми з обладнанням чи ПЗ	Середня	Високий	Зниження (регулярні оновлення)

Висновки до розділу 2

У даному розділі було проведено планування процесу розробки вебзастосунку LibProj. За результатами оцінки тривалості з використанням методу Use Case Points визначено загальний обсяг робіт, а декомпозиція проєкту за методологією WBS дозволила структурувати виконання на сім основних етапів загальною тривалістю близько 170 робочих днів (01.09.2025 – 30.04.2026). Розроблена діаграма Ганта демонструє чітку послідовність фаз із частковим перекриттям: специфікація вимог (40 р.д.), планування розробки (30 р.д.), проєктування (50 р.д.), програмна реалізація (72 р.д.), тестування функціональності (45 р.д.), експлуатація (25 р.д.) та підготовка звіту (40 р.д.). Паралельне виконання фаз проєктування, реалізації та тестування забезпечує оптимізацію загального терміну проєкту без втрати якості.

Проведений аналіз ризиків виявив, і для кожного ідентифікованого ризику визначено ймовірність виникнення, потенційний вплив та розроблено превентивні заходи і плани реагування. Таким чином, проведене планування створює надійну основу для успішної реалізації, забезпечуючи реалістичні терміни виконання, чітку послідовність робіт та готовність до потенційних викликів під час розробки.

3 ПРОЄКТУВАННЯ ВЕБЗАСТОСУНКУ

3.1 Визначення архітектури застосунку

Архітектура вебзастосунку LibProj побудована за модульною трирівневою клієнт-серверною архітектурою з використанням патерну App Router, що поєднує клієнтський та серверний рендеринг в єдиному застосунку. Ключовою особливістю є використання патерну Server Components + Server Actions, який дозволяє виконувати серверну логіку без створення окремого API-сервера – серверний код викликається напряму з React-компонентів через RPC-подібний механізм.

Компоненти розділені на серверні, що рендеряться на сервері та мають прямий доступ до бази даних, та клієнтські, що обробляють інтерактивність у браузері. Бізнес-логіка зосереджена у Server Actions – асинхронних серверних функціях, позначених директивою 'use server', що розташовані в директорії actions. Server Actions викликаються безпосередньо з клієнтських компонентів через useFormAction або прямий виклик функції – фреймворк автоматично серіалізує параметри, виконує функцію на сервері та повертає результат клієнту. Для HTTP-інтеграцій зі сторонніми сервісами використовуються API Routes.

Доступ до бази даних реалізований через ORM із єдиним екземпляром клієнта (Singleton-патерн). ORM забезпечує типобезпечні запити на основі схеми, яка визначає понад 20 моделей даних. База даних є реляційною (PostgreSQL) з підтримкою serverless-підключень.

Архітектура інтегрує зовнішні сервіси через відповідні рівні абстракції:

- рівень автентифікації (хешування паролів, JWT-сесії, контроль доступу за ролями);
- рівень платежів (створення та захоплення платіжних транзакцій);
- рівень пошуку метаданих (отримання інформації про книги за ISBN, назвою, автором);
- рівень файлового сховища (завантаження зображень обкладинок);

– рівень AI (генерація описів книг та резюме відгуків на основі мовних моделей).

На рис. 3.1 представлена загальна архітектура вебзастосунку.

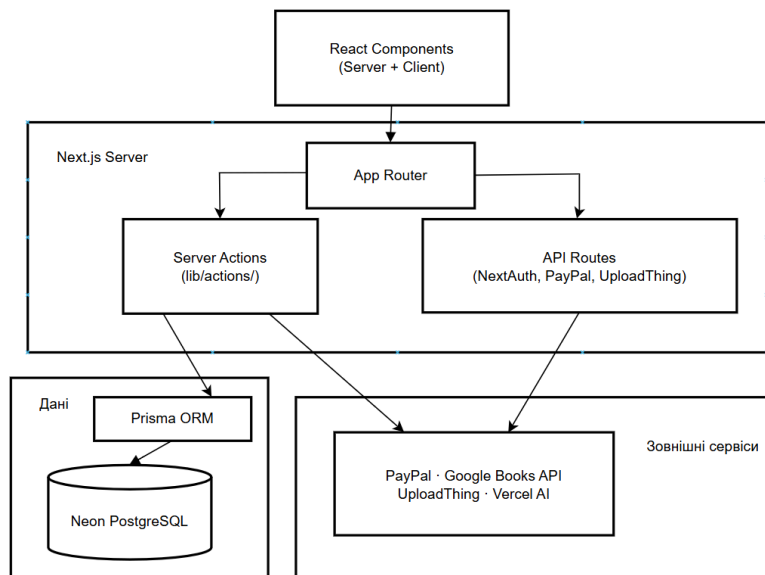


Рисунок 3.1 – Загальна архітектура

3.2 Опис та проектування програмних класів

Замість об'єктно-орієнтованого підходу з класами, архітектура застосунку побудована на основі функціональних модулів. Кожен модуль інкапсулює логіку, пов'язану з конкретною функціональністю.

AuthenticationModule відповідає за всі аспекти аутентифікації та авторизації. Цей модуль забезпечує безпечну реєстрацію користувачів, вхід з перевіркою даних, управління сесіями та контроль доступу до різних функцій на основі ролей.

BookCatalogModule є компонентом для управління каталогом мистецької літератури. Модуль надає функціональність перегляду детальної інформації, включаючи опис, рейтинг та відгуки. Особливістю є можливість додавання нових книг, а також генерація описів книг на основі наявних метаданих.

CartOrderModule забезпечує повний цикл здійснення покупок літератури.

Модуль управляє кошиком покупок, дозволяючи додавати, видаляти та змінювати кількість, автоматично розраховує загальну вартість та доставки.

EventClubModule компонент для організації літературних подій та читацьких клубів. Модуль дозволяє користувачам подавати заявки на створення клубів або подій.

ReviewModule надає користувачам можливість поділитися своїми враженнями про прочитане. Модуль підтримує написання рецензій з оцінками, деревоподібні відповіді на відгуки, голосування за корисність відгуків, подання скарг на неприйнятний контент та генерацію AI-резюме відгуків.

AnalyticsModule збирає та аналізує агреговані дані для адміністраторів. Модуль генерує звіти за категоріями: тренди читання, залученість спільноти, активність подій та клубів, поведінка користувачів. Кожен звіт має статус виконання та підтримує різні періоди аналізу.

PaymentModule відповідає за безпечну обробку транзакцій. Модуль інтегрується з платіжною системою для прийому платежів.

AdminModule надає інструменти для управління. Функціональність включає додавання нових книг до каталогу та редагування існуючих, управління замовленнями, модерацію контенту, генерацію звітів.

SearchModule забезпечує швидкий пошук контенту. Модуль використовує повнотекстовий пошук для знаходження книг, подій та клубів, підтримує розширений пошук з множинними фільтрами.

PersonalAnalyticsModule відповідає за персональну аналітику читання. Модуль дозволяє встановлювати цілі читання на рік, відстежувати серії активності, отримувати досягнення за активність та налаштовувати приватність аналітичного профілю.

BookSubmissionModule забезпечує процес подання книг користувачами до каталогу. Модуль включає автозаповнення метаданих через Google Books API за ISBN-кодом, перевірку дублікатів пошук по існуючому каталогу, генерацію описів за допомогою AI та передачу поданих книг на модерацію адміністратору.

На рис. 3.2 представлено діаграму класів з усіма модулями та їх взаємозв'язками.

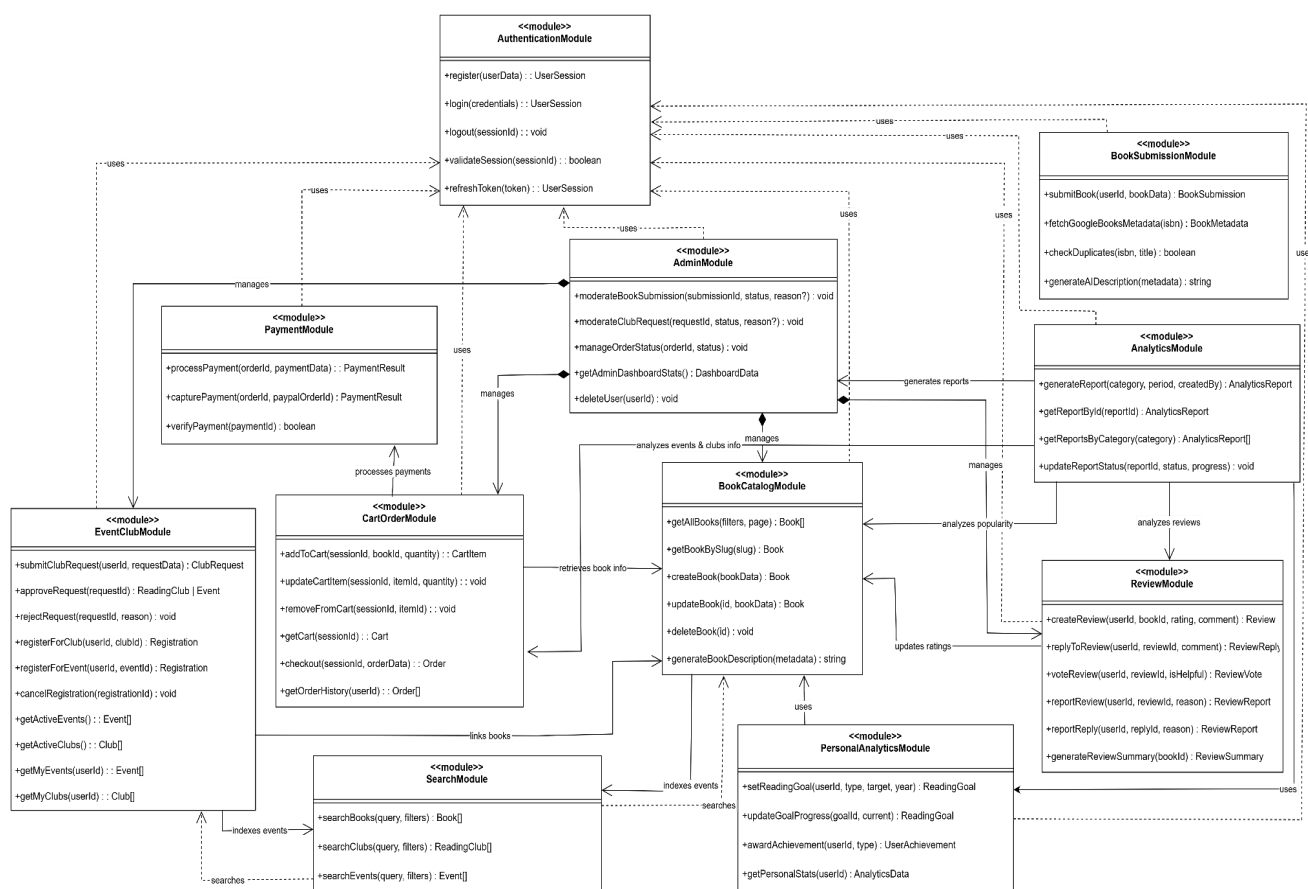


Рисунок 3.2 – Діаграма класів

AuthenticationModule модуль, від якого залежать інші. BookCatalogModule взаємодіє з SearchModule для індексації нових книг. ReviewModule звертається до BookCatalogModule для оновлення рейтингів на основі нових. CartOrderModule отримує інформацію з BookCatalogModule при формуванні замовлень. CartOrderModule інтегрований з PaymentModule для обробки платежів. EventClubModule використовує BookCatalogModule для прив'язки книг до подій та клубів. AdminModule має доступ до всіх основних модулів для модерації каталогу, затвердження подій та клубів, генерації звітів. AnalyticsModule агрегує дані з CartOrderModule, EventClubModule та ReviewModule. PersonalAnalyticsModule використовує дані з ReviewModule, EventClubModule та CartOrderModule для розрахунку персональних цілей, серій та досягнень.

3.3 Структура бази даних

Розглянемо таблиці основних сутностей та створимо загальну схему усіх таблиць. Першу маємо таблицю User, яка є центральною (табл. 3.1).

Таблиця 3.1 – Схема таблиці User

Назва поля	Тип даних	Ключ	Призначення
id	UUID	primary	Унікальний ідентифікатор
name	String	-	Ім'я
email	String	-	Електронна пошта
password	String	-	Пароль
role	String	-	Роль
createdAt	DateTime	-	Дата додавання

Наступною розглянемо таблицю Book, що формує каталог книг (табл. 3.2).

Таблиця 3.2 – Схема таблиці Book

Назва поля	Тип даних	Ключ	Призначення
id	UUID	primary	Унікальний ідентифікатор
name	String	-	Назва
slug	String	unique	URL-сегмент
category	String	-	Категорія
price	Decimal(12,2)	-	Ціна
author	String	-	Автор
rating	Decimal(3,2)	-	Середній рейтинг
stock	Integer	-	Кількість (якщо є)

Далі розглянемо таблицю Review, що відображає відгуки користувачів на літературу. Кожен відгук містить текстовий коментар та числову оцінку (табл. 3.3).

Таблиця 3.3 – Схема таблиці Review

Назва поля	Тип даних	Ключ	Призначення
id	UUID	primary	Ідентифікатор звіту
userId	UUID	FK (User)	Автор відгуку
bookId	UUID	FK (Book)	Книга, до якої відгук
rating	Int	–	Оцінка (1–5)
comment	String	–	Текст відгуку
createdAt	DateTime	–	Дата створення

Наступною представлено таблицю Order, представляє замовлення на придбання літератури. Відстежує статус оплати та доставки (табл. 3.4).

Таблиця 3.4 – Схема таблиці Order

Назва поля	Тип даних	Ключ	Призначення
id	UUID	PK	Унікальний ідентифікатор замовлення
userId	UUID	FK (User)	Покупець
totalPrice	Decimal(12,2)	–	Загальна сума
isPaid	Boolean	–	Чи оплачено
isDelivered	Boolean	–	Чи доставлено
createdAt	DateTime	–	Дата створення

Наступною маємо схему таблиці ClubRequest, що відображає заявки користувачів на створення читацького клубу або події (табл. 3.5). Кожна заявка проходить модерацию адміністратором і може отримати статус pending, approved або rejected.

Після схвалення заявка автоматично створює відповідну сутність – ReadingClub або Event – через зв'язок один-до-одного.

Таблиця 3.5 – Схема бази даних таблиці ClubRequest

Назва поля	Тип даних	Ключ	Призначення
id	UUID	PK	Унікальний ідентифікатор заявки
userId	UUID	FK (User)	Автор заявки
type	Enum	–	Тип: club або event
title	String	–	Назва
status	Enum	–	Статус
format	Enum	–	Онлайн/офлайн
capacity	Int	–	Кількість учасників

Розглянемо таблицю ReadingClub. Читацькі клуби створені після схвалення заявки. Об'єднують учасників для спільного обговорення (табл. 3.6).

Таблиця 3.6 – Схема бази даних таблиці ReadingClub

Назва поля	Тип даних	Ключ	Призначення
id	UUID	primary	Унікальний ідентифікатор
clubRequestId	UUID	foreign/unique	ID схваленого запиту
title	String	–	Назва клубу
description	String	–	Опис клубу
capacity	Integer	–	Кількість учасників
format	ClubEventFormat	–	Формат проведення
bookIds	String[]	–	Масив ID літератури
creatorId	UUID	foreign	Ідентифікатор
isActive	Boolean	–	Активність клубу

Отже, тепер можемо спроектувати повну схему бази даних, включаючи усі існуючі таблиці, зображену на рис. 3.3.

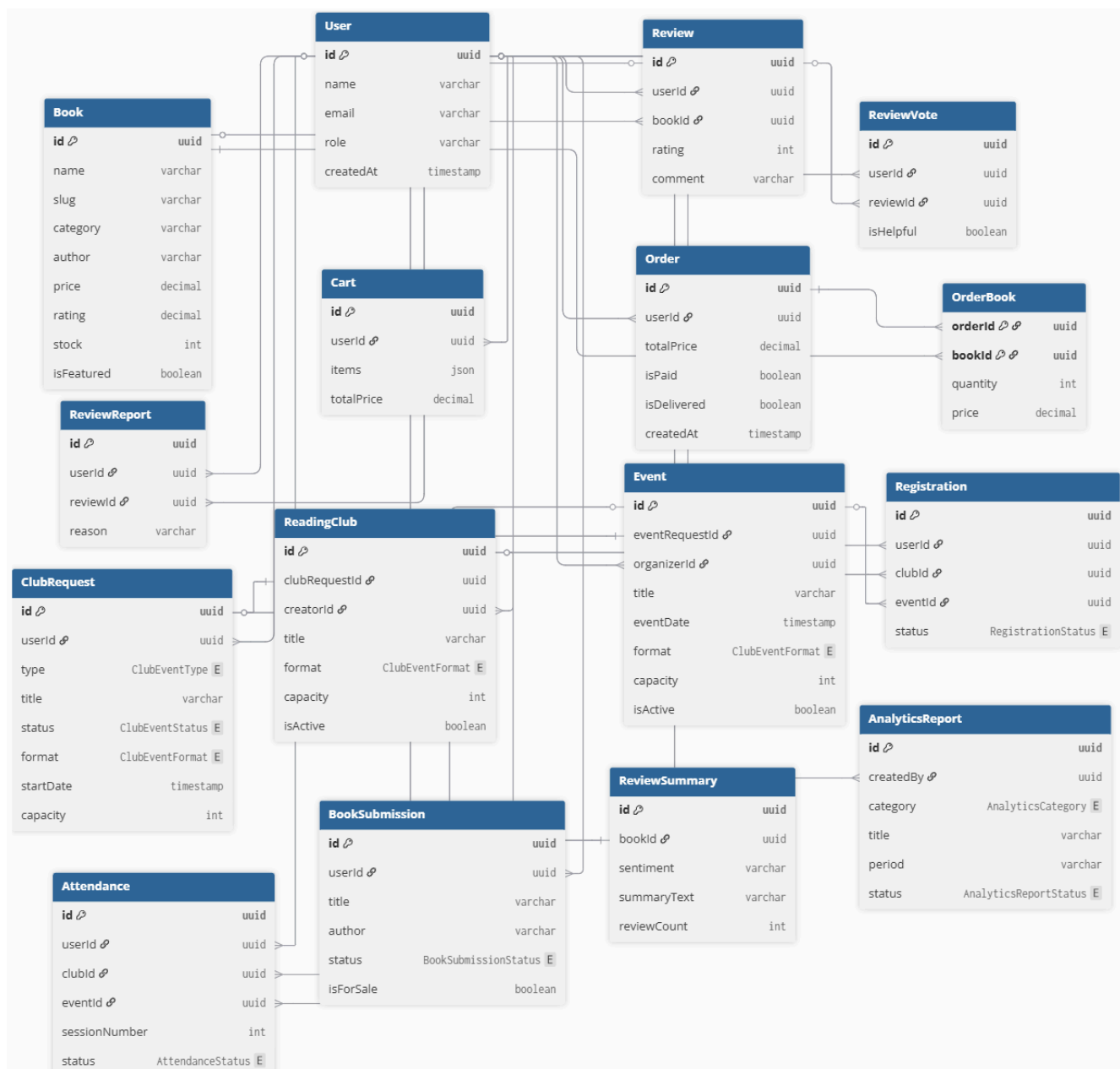


Рисунок 3.3 – Схема бази даних

User – центральна таблиця. Має зв'язок один-до-багатьох із: Review, ReviewVote, ReviewReport, Cart, Order, ClubRequest, ReadingClub (як creator), Event (як organizer), Registration, Attendance, BookSubmission, AnalyticsReport.

Book має багато Review та OrderBook (один-до-багатьох), а також рівно один ReviewSummary (один-до-одного) – AI-резюме відгуків.

Review належить User і Book. Має багато ReviewVote та ReviewReport.

ReviewVote та ReviewReport належать User і Review (один-до-багатьох від Review).

Order належить User і містить багато OrderBook. Кожен OrderBook посилається на Order та Book (складений первинний ключ orderId + bookId).

Cart належить User (один-до-багатьох) і зберігає товари у JSON-полі.

ClubRequest належить User. Після схвалення створює рівно один ReadingClub або один Event (зв'язок один-до-одного).

ReadingClub і Event мають однакову структуру зв'язків: кожен має багато Registration та Attendance.

Registration і Attendance належать User і посилаються на ReadingClub або Event (обидва поля опціональні – заповнюється одне з двох).

BookSubmission належить User – це заявка на додавання книги в каталог.

AnalyticsReport належить User (один-до-багатьох) – звіти аналітики.

ReviewSummary має зв'язок один-до-одного з Book (унікальний bookId).

Усі зв'язки мають каскадне видалення (onDelete: Cascade) – при видаленні батьківського запису дочірні видаляються автоматично.

3.4 Проєктування процесів

Проєктування процесів дозволяє візуалізувати взаємодію між компонентами та послідовність виконання операцій. Для опису ключових процесів платформи використовуються діаграми послідовності[6], які демонструють ключові аспекти роботи.

Розглянемо процес додавання книги користувачем. Діаграма послідовності додавання книги демонструє взаємодію між користувачем, інтерфейсом, серверними компонентами та базою даних. Процес охоплює етапи від ініціювання запиту користувачем до підтвердження успішного збереження даних. Кожен крок супроводжується валідацією введених даних та відповідними повідомленнями про статус виконання операції.

На рис. 3.4 представлено дану діаграму послідовності.

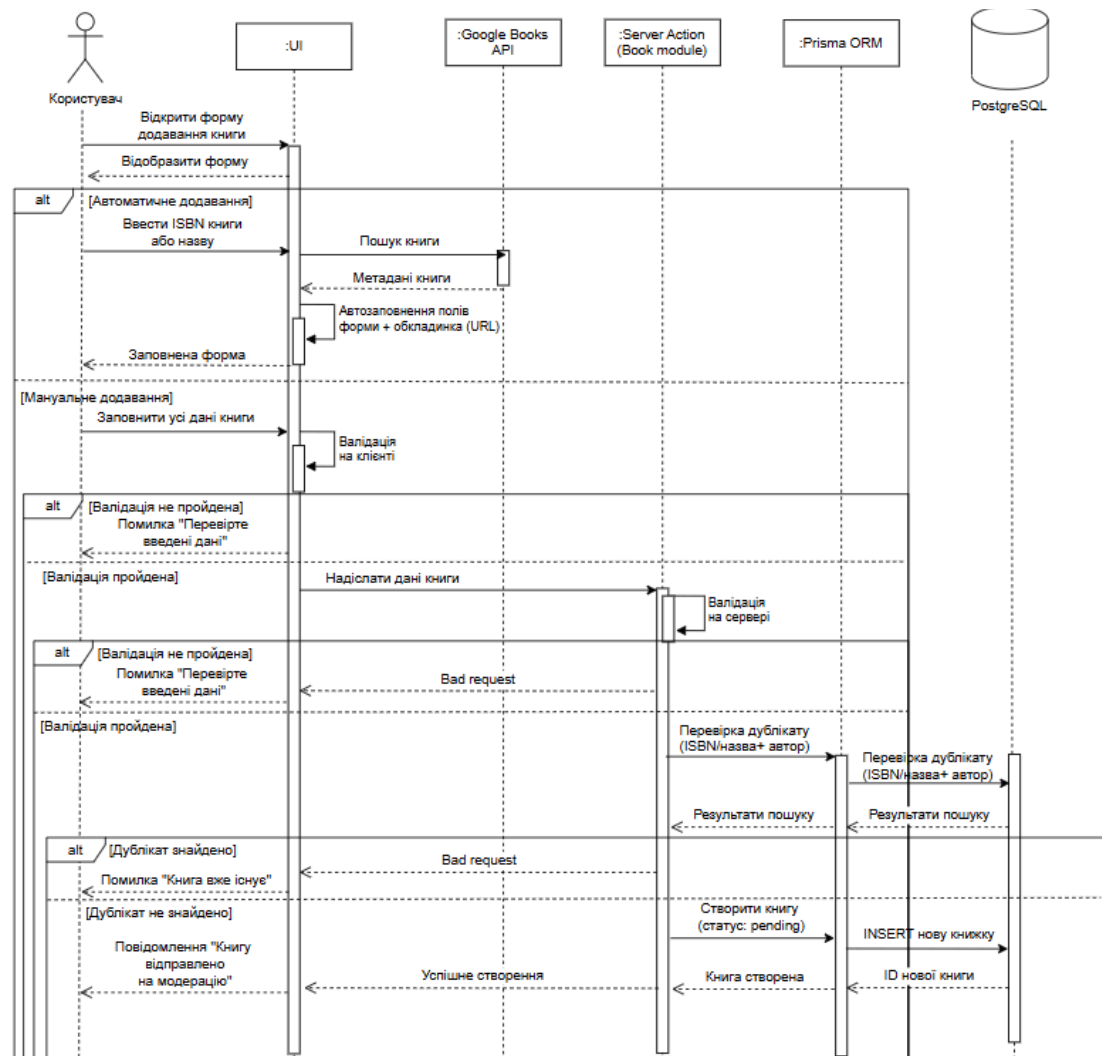


Рисунок 3.4 – Діаграма діяльності процесу додавання книги користувачем

Наступною представимо діаграму діяльності, яка демонструє інтеграцію з AI сервісом для автоматичної генерації описів книг на основі введених метаданих.

Промпт включає в себе:

- метадані книги;
- правила тону (неформальний/академічний/реklamний);
- обмеження довжини (короткий/середній/довгий);
- цільова мова;
- заборони (без повторення заголовків, без шаблонних вступів).

Даний етап не є обов'язковим під час створення запиту на додавання нової книги до каталогу, та виконується під час заповнення форми перед відправкою даних. Розглянемо окремо виділену діаграму діяльності на рис. 3.5.

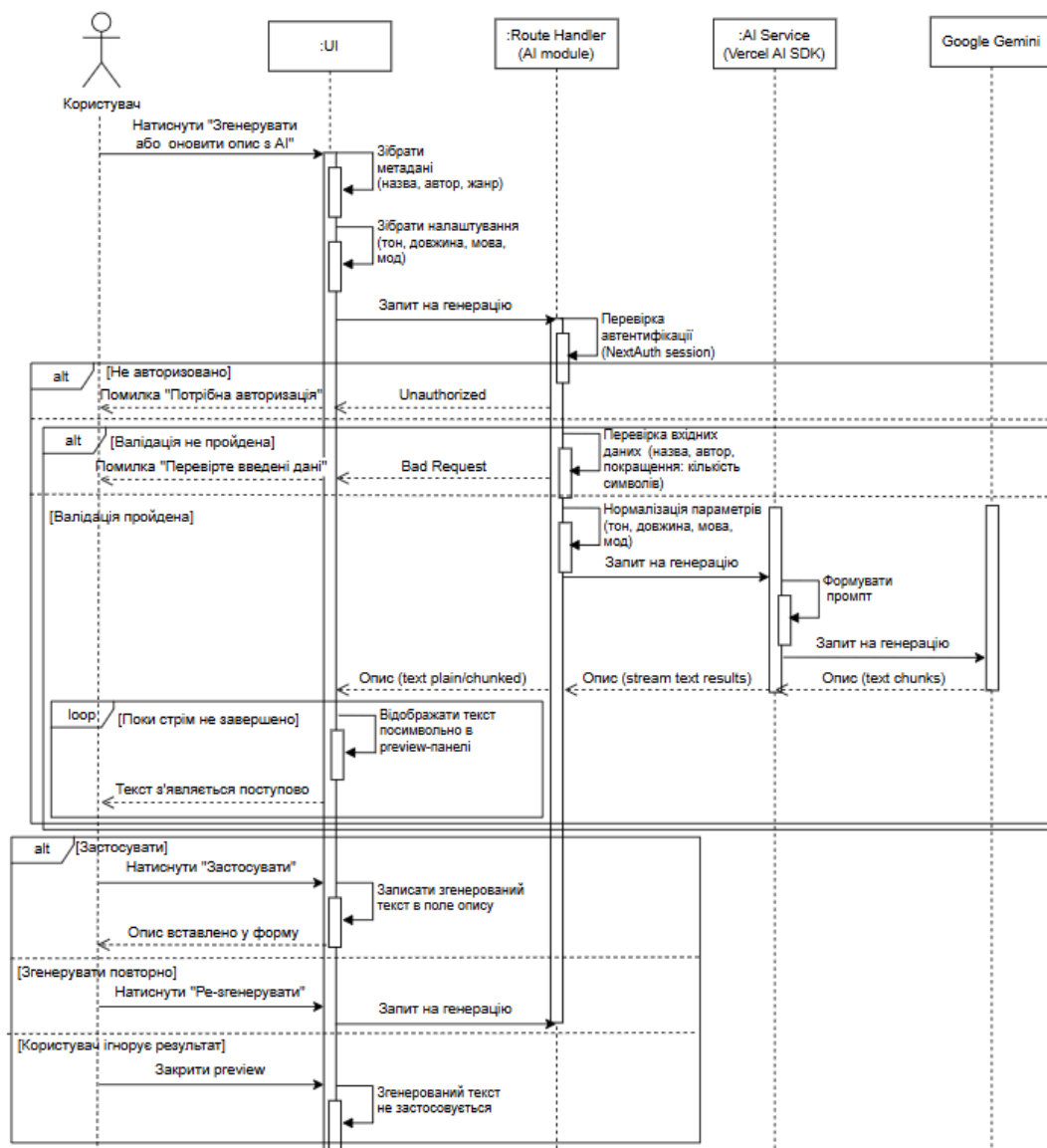


Рисунок 3.5 – Діаграма діяльності процесу генерації опису за допомогою AI

Розглянемо процес генерації аналітики адміністратором. Діаграма послідовності генерації аналітики демонструє взаємодію між адміністратором, інтерфейсом адміністративної панелі, серверними компонентами та базою даних. Процес включає запит агрегованих статистичних даних по книгах, користувачах та активності читання, їх обробку на сервері та візуалізацію у вигляді дашборду з картками показників та графіками трендів. Адміністратор має можливість

фільтрувати дані за обраним періодом. На рис. 3.6 представлено дану діаграму послідовності.

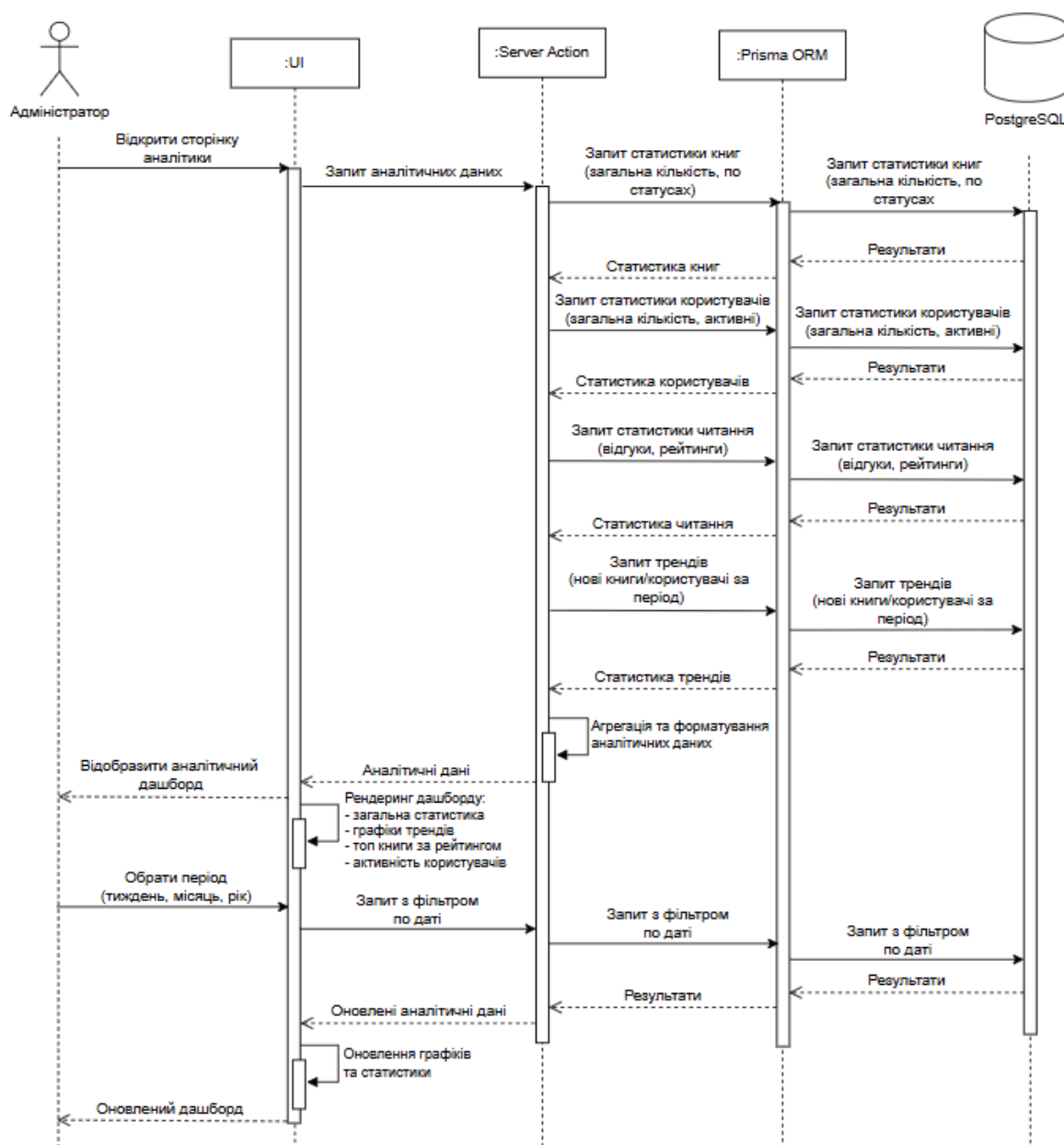


Рисунок 3.6 – Діаграма діяльності процесу генерації аналітики адміністратором

3.5 Проектування інтерфейсу користувача

Проектування інтерфейсу користувача є важливим етапом розробки, що визначає зручність та ефективність взаємодії з платформою. При проектуванні інтерфейсу було дотримано принципів простоти навігації, мінімалізму та

доступності. Розглянемо макети ключових сторінок, що відображають основні сценарії. Макет сторінки каталогу мистецької літератури (рис. 3.7) є основною точкою входу для пошуку. Сторінка буде містити панель пошуку з можливістю фільтрації за жанром, рейтингом та статусом, а також сітку карток книг.

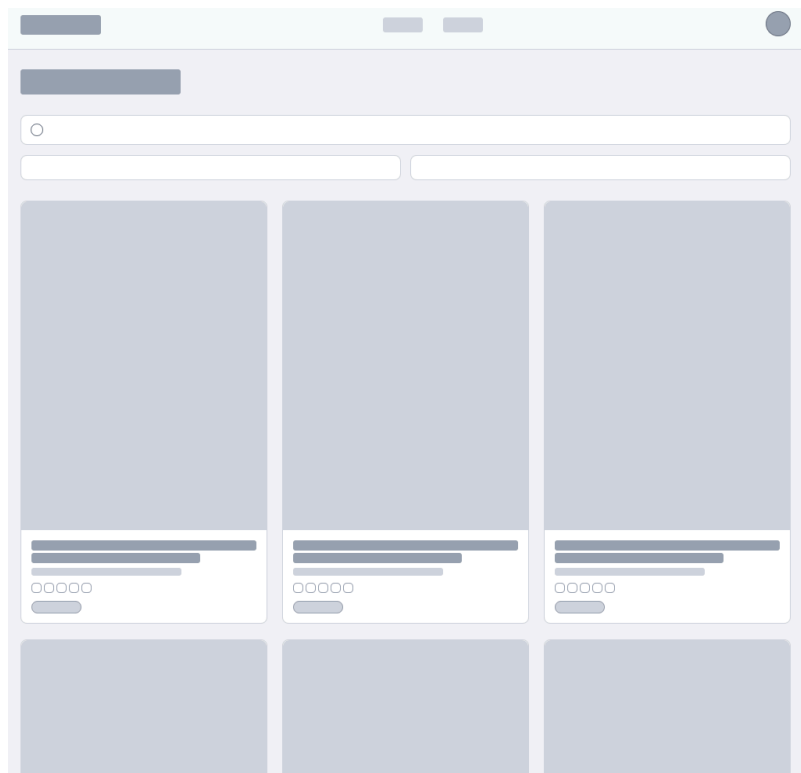


Рисунок 3.7 – Макет сторінки каталогу літератури

Форма додавання нової книги до каталогу літератури буде інтегрована з Google Books API, і при введенні ISBN або назви книги поля автоматично заповнюються отриманими метаданими.

Окремим елементом інтерфейсу буде компонент для генерації опису, який дозволяє згенерувати або покращити опис книги за допомогою штучного інтелекту з вибором тону, довжини та мови генерації.

Згенерований текст відображатиметься у панелі попереднього перегляду в реальному часі та буде застосовуватись до форми лише після підтвердження користувачем.

Макет форми додавання нової книги до каталогу літератури представлений на рис. 3.8.

 A wireframe of a form for adding a new book. It features a vertical sidebar on the left with several menu items. The main area contains a series of horizontal input fields for text, followed by three radio buttons. Below these is a large rectangular text area. At the bottom, there is a dropdown menu and a prominent button.

Рисунок 3.8 – Макет форми додавання літератури

Макет панелі модерації адміністратора (рис. 3.9) необхідний для управління контентом вебзастосунку. Сторінка буде містити таблицю книг зі статусом «очікує модерації» з можливістю перегляду деталей, схвалення або відхилення кожного запису. Адміністратор буде мати змогу фільтрувати записи за статусом та виконувати масові дії.

 A wireframe of an administrator moderation panel. It includes a sidebar with navigation links. The main content area features a table with 10 rows, each representing a book record. Each row contains a checkbox, a small thumbnail, and several text fields. To the right of each row are three icons for actions: a magnifying glass (details), a checkmark (approve), and an 'X' (reject). Above the table is a search bar and a filter dropdown. At the bottom right, there are buttons for 'Add' and 'Refresh'.

Рисунок 3.9 – Макет панелі керування адміністратора

Аналітичний дашборд (рис. 3.10) надасть адміністратору візуалізацію ключових показників платформи. Верхня частина буде містити картки зведеної статистики. Нижче будуть розміщені графіки трендів активності за обраний період – динаміка додавання книг, реєстрації користувачів та залишених відгуків.

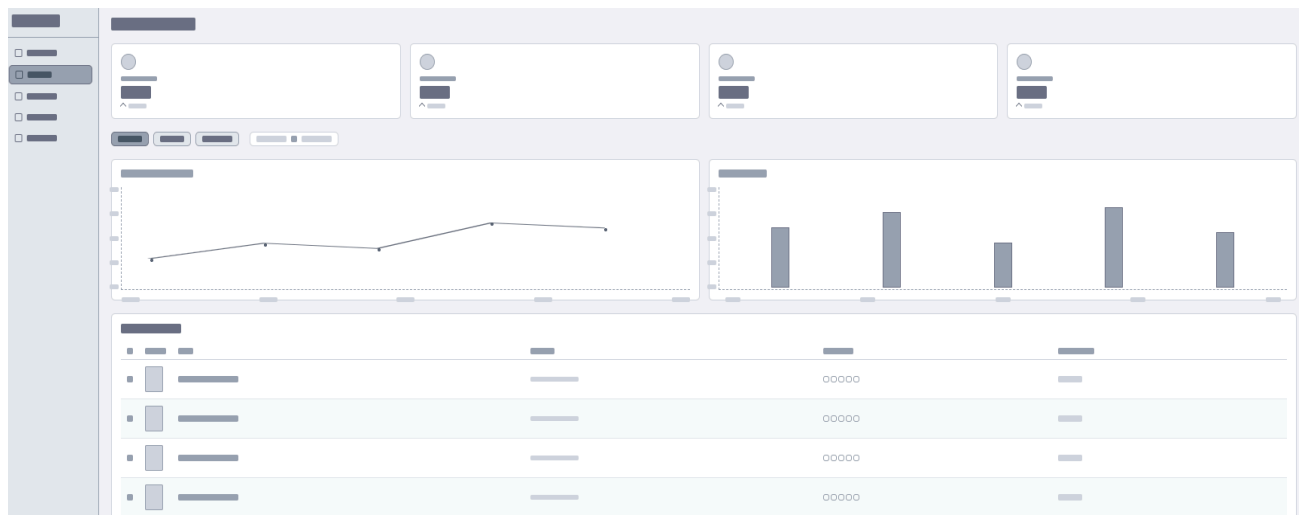


Рисунок 3.10 – Макет сторінки аналітичного дашборду

Висновки до розділу 3

У третьому розділі виконано проєктування вебзастосунку на основі функціональних модулів. Розроблено 11 модулів, що інкапсулюють бізнес-логіку конкретних предметних областей та мають чітко визначені інтерфейси взаємодії. Спроектовано нормалізовану структуру бази даних із 16 таблиць, оптимальними зв'язками та каскадним видаленням для забезпечення цілісності даних.

Розроблено діаграми діяльності для ключових процесів: додавання книги з автозаповненням через Google Books API, генерація описів за допомогою AI та формування аналітичних звітів. Спроектовано інтерфейс користувача з дотриманням принципів мінімалізму та доступності – створено макети каталогу, форми додавання книги, панелі модерації та аналітичного дашборду.

Результати проєктування забезпечують надійну основу для реалізації програмного продукту.

4 ПРОГРАМНА РЕАЛІЗАЦІЯ ВЕБЗАСТОСУНКУ

4.1 Вибір інструментів розробки

При виборі технологічного стеку для розробки було проведено аналіз сучасних інструментів та фреймворків. Основним критерієм вибору стала необхідність створення масштабованого та зручного для користувачів вебзастосунку з можливістю подальшого розширення функціональності.

Для реалізації вебзастосунку було обрано Next.js версії 15[7], який представляє собою повноцінне рішення для створення React-застосунків з підтримкою серверного рендерингу. Цей вибір обумовлений необхідністю забезпечити високу швидкість завантаження сторінок і вбудовану підтримку Server Actions, що усуває потребу у створенні окремого бекенд-сервера.

Для забезпечення типобезпеки використовується TypeScript[8]. Статична типізація дозволяє виявляти помилки на етапі розробки.

Стилізація інтерфейсу реалізована за допомогою бібліотеки компонентів MUI для швидкого створення адаптивних дизайнів.

Для роботи з базою даних використовується Prisma ORM[9] у поєднанні з Neon Database. Prisma забезпечує типобезпечні запити до бази даних та зручний API для роботи з даними. Neon Database було обрано як сучасну serverless PostgreSQL[10] базу даних, що забезпечує автоматичне масштабування і оптимальну продуктивність без необхідності управління інфраструктурою.

Для системи автентифікації було обрано NextAuth.js версії 5[11]. Ця бібліотека надає готове рішення для реалізації різних стратегій авторизації – вхід через облікові дані з bcrypt-хешуванням паролів, підтримка OAuth-провайдерів, автоматичне управління JWT-сесіями та refresh-токенами.

Інтегровано Vercel AI SDK[12], який надає інтерфейс для роботи з різними AI-моделями. Це дозволяє реалізувати функції автоматичної генерації описів книг та AI-резюме відгуків.

Обробка платежів реалізована через PayPal REST API[13] з використанням React SDK, що забезпечує зручну інтеграцію платіжної системи. Бібліотека надає готові компоненти для відображення кнопок оплати та обробки транзакцій.

Google Books API[14] використовується для автозаповнення інформації за ISBN-кодом, назвою або автором, що спрощує процес подання літератури.

UploadThing[15] забезпечує завантаження зображень обкладинок книг з автоматичною оптимізацією та CDN-доставкою.

4.2 Технологія створення вебзастосунку

Програмна реалізація вебзастосунку охоплює декілька ключових процесів, розглянемо найосновніші з них: наповнення каталогу мистецькою літературою, AI-генерація текстового контенту та агрегація аналітичних даних.

Каталог наповнюється двома шляхами: подання книги користувачем (BookSubmission – модерація – Book) та пряме додавання адміністратором. При поданні книги валідація здійснюється через Zod-схему[16] bookSubmissionSchema, яка перевіряє: назву, автора, опис, наявність хоча б однієї категорії, а також умовно-обов'язкове поле suggestedPrice при isForSale=true (див. лістинг 4.1).

Лістинг 4.1 – Код фрагменту Zod-схеми валідації подання книги

```
export const bookSubmissionSchema = z.object({
  title: z.string().min(1).max(255),
  author: z.string().min(1).max(128),
  isbn: z.string().optional().nullable().refine((val) => {
    if (!val) return true;
    return /^(\\d{10}|\\d{13})$/.test(val.replace(/[-\\s]/g, ''));
  }, { message: 'ISBN must be 10 or 13 digits' }),
  description: z.string().min(10).max(2000),
  categories: z.array(z.string()).min(1, 'At least one category is required'),
  isForSale: z.boolean().default(false),
  suggestedPrice: z.coerce.number().min(0.01).max(99999).optional().nullable(),
}).refine((data) => {
  if (data.isForSale && (!data.suggestedPrice || data.suggestedPrice <= 0))
    return false;
  return true;
}, { message: 'Price is required when the book is for sale', path:
['suggestedPrice'] });
```

Інтеграція з Google Books API реалізована через три функції: для точного пошуку за ISBN-кодом, для пошуку за метаданими, та для отримання деталей конкретного видання. Внутрішня функція `parseGoogleBookItem()` нормалізує відповідь API: витягує ISBN-10/ISBN-13, обирає обкладинку найвищої якості, конвертує HTTP-посилання на HTTPS та визначає мову через `detectLanguageFromTitle()` за Unicode-діапазонами символів (див. лістинг 4.2).

Лістинг 4.2 – Код фрагменту нормалізації відповіді Google Books API

```
function parseGoogleBookItem(item: GoogleBookItem): ParsedBookData {
  const { volumeInfo } = item;
  let isbn: string | undefined, isbn13: string | undefined;
  for (const id of volumeInfo.industryIdentifiers || []) {
    if (id.type === 'ISBN_10') isbn = id.identifier;
    else if (id.type === 'ISBN_13') isbn13 = id.identifier;
  }
  const coverImage = (volumeInfo.imageLinks?.extraLarge ||
    volumeInfo.imageLinks?.large || volumeInfo.imageLinks?.medium ||
    volumeInfo.imageLinks?.thumbnail)?.replace('http://', 'https://');
  return {
    title: volumeInfo.title + (volumeInfo.subtitle ? `: ${volumeInfo.subtitle}` :
    ''),
    author: volumeInfo.authors?.join(', ') || 'Unknown Author',
    isbn, isbn13, coverImage, googleBooksId: item.id,
    categories: volumeInfo.categories || [],
  };
}
```

Для запобігання дублікатам реалізовано `debounced`-пошук через `searchCatalog()`, що виконує порівняння введеної назви з існуючими записами в таблиці `Book`.

Генерація описів реалізована через Vercel AI SDK з моделлю Gemini 2.5 Flash (температура: 0.7, максимальна кількість токенів на виході: 4096). API-ендпоінт `/api/generate-book-description` приймає POST-запит з метаданими книги та параметрами генерації: тон (академічний, звичайний, рекламний), довжина (коротка, середня, довга), мод (генерація, покращення). У режимі генерації промпт формується динамічно з урахуванням жанру, мови та доступних метаданих. У режимі покращення модель отримує існуючий опис із завданням покращити текст (див. лістинг 4.3).

Лістинг 4.3 – Код фрагменту генерації опису книги

```
const TONE_GUIDE: Record<DescriptionTone, string> = {
  academic: 'Use a formal, scholarly tone. Focus on thematic depth.',
  casual: 'Use a warm, conversational tone as if recommending to a friend.',
  promotional: 'Use compelling, marketing-oriented language.',
};
const LENGTH_GUIDE: Record<DescriptionLength, string> = {
  short: '2-3 sentences (50-80 words)',
  medium: '1-2 paragraphs (120-200 words)',
  long: '2-3 paragraphs (250-400 words)',
};

export function generateBookDescription(params: GenerateDescriptionParams) {
  return streamText({
    model: google('gemini-2.5-flash'),
    prompt: buildPrompt(params),
    maxOutputTokens: 4096, temperature: 0.7,
  });
}
```

Модуль аналітики адміністратора реалізує агрегацію даних через сирі SQL-запити для максимальної продуктивності. Дані агрегуються за чотирма категоріями: тренди читання (динаміка продажів, топ-10 категорій за виручкою, найпопулярніша література, тренд відгуків), залученість спільноти (активність рецензій, топ-рецензенти), події та читацькі клуби (розподіл за форматом, топ-клуби за кількістю учасників), поведінка користувача (реєстрації за день, теплова карта активності (день тижня × година), топ-покупці за сумою витрат) (див. лістинг 4.4).

Лістинг 4.4 – Код фрагменту SQL-агрегації трендів читання

```
export async function getReadingTrends(period: string, customStart?: string,
customEnd?: string) {
  const { start, end } = dateRange(period, customStart, customEnd);
  const dailyOrders: DateCountRevenue[] = await prisma.$queryRaw`
    SELECT DATE("createdAt") as date, COUNT(*)::bigint as count,
    FROM "Order"
    WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
    GROUP BY DATE("createdAt") ORDER BY date`;
  const topCategories: CategoryRevenue[] = await prisma.$queryRaw`
    SELECT p."category", SUM(oi."price" * oi."quantity") as revenue
    FROM "OrderItem" oi JOIN "Product" p ON p."id" = oi."productId"
    JOIN "Order" o ON o."id" = oi."orderId"
    WHERE o."createdAt" >= ${start} AND o."createdAt" <= ${end}
    GROUP BY p."category" ORDER BY revenue DESC LIMIT 10`;
  const buckets = generateDailyBuckets(start, end);
  return { salesTrend: buckets.map(d => ({ date: d, orders: ... })), topCategories
};
```

Функція `generateAnalyticsReport()` створює запис у таблиці `AnalyticsReport` зі статусом `processing`, поетапно оновлює прогрес (10% – 30% – 100%), виконує відповідну агрегацію та зберігає результат у полі `data` (тип `JSON`). При помилці статус змінюється на `failed` (див. лістинг 4.5).

Лістинг 4.5 – Код фрагменту автоматизації генерації аналітичного звіту

```
export async function generateAnalyticsReport(input: {
  category: 'reading_trends' | 'community_engagement' | 'events_clubs' |
  'user_behavior';
  period: string; customStart?: string; customEnd?: string;
}) {
  const session = await auth();
  if (session?.user?.role !== 'admin') throw new Error('Not authorized');
  const { start, end } = dateRange(input.period, input.customStart,
input.customEnd);
  const report = await prisma.analyticsReport.create({ data: {
    category: input.category, period: input.period,
    startDate: start, endDate: end, data: {}, status: 'processing', progress: 10,
    title: `${input.category.replace(/_/g, ' ')} report`, createdBy:
session.user.id!,
  }});
  try {
    let data = await getDataByCategory(input.category, input.period);
    await prisma.analyticsReport.update({ where: { id: report.id },
      data: { data: data as Prisma.InputJsonValue, status: 'completed', progress:
100 }});
  } catch { await prisma.analyticsReport.update({ where: { id: report.id },
    data: { status: 'failed', progress: 0 }}); }
}
```

4.3 Приклади роботи вебзастосунку

Розглянемо детальні приклади функціональності вебзастосунку `LibProj` через опис кроків взаємодії з користувачем. Кожен сценарій охоплює послідовність дій від початкового звернення до системи до отримання кінцевого результату.

Почнемо з формування каталогу мистецької літератури та розглянемо повний процес його наповнення – від подання книги користувачем до її публікації адміністратором у загальнодоступному каталозі. Цей процес передбачає кілька етапів: заповнення форми з інформацією про видання, проходження модерації та остаточне затвердження, після якого книга стає доступною для перегляду, пошуку та замовлення іншими користувачами.

На рис. 4.1 представлений алгоритм формування каталогу.

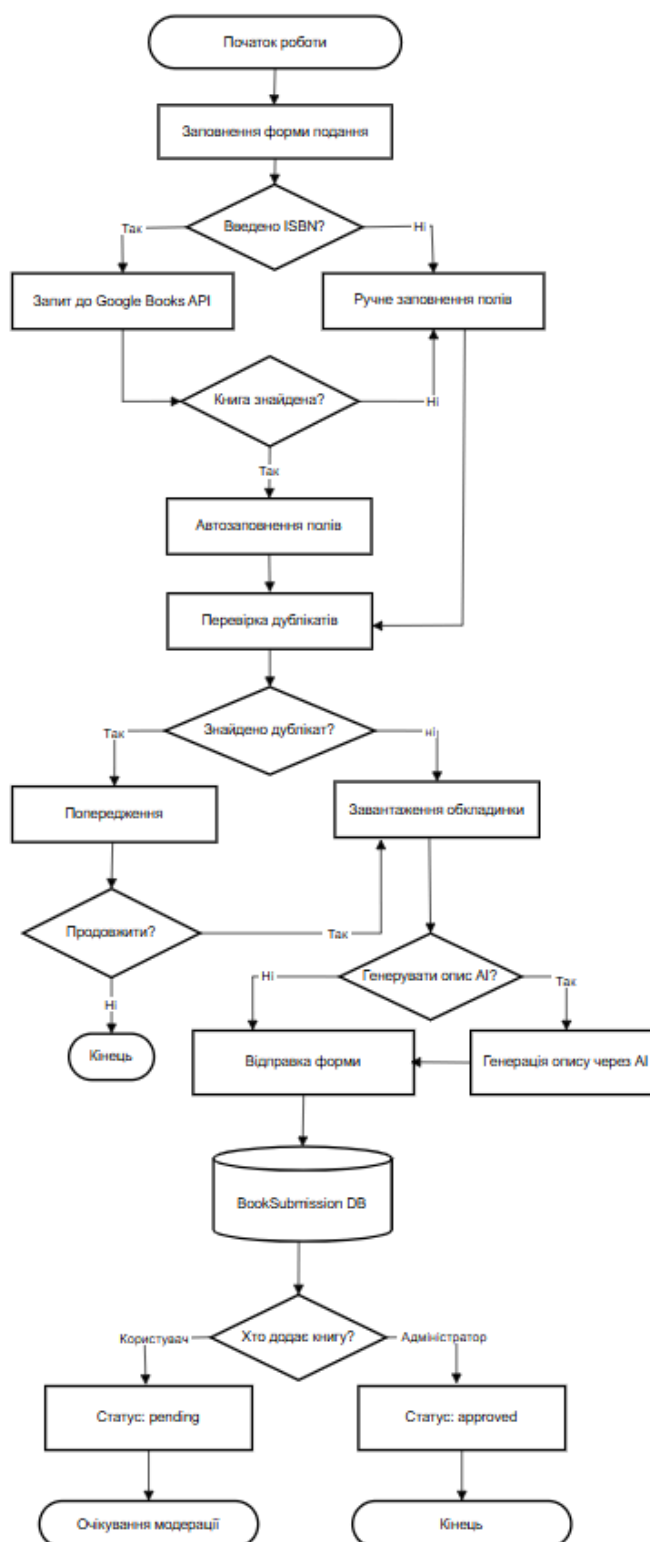


Рисунок 4.1 – Блок схема процесу формування каталогу

Для подання нової книги до каталогу користувач переходить на сторінку через навігаційне меню.

Вебзастосунок відображає форму подання книги з усіма необхідними полями, яка представлена на рис. 4.2.

Submit a New Book
Can not find a book in our catalog? Submit it here and we will add it after review.

Quick Fill with ISBN
Enter an ISBN to automatically fill book details from Google Books

Enter ISBN (10 or 13 digits)

Or Search Google Books
Search by title and author to find your book

Book Details

Title *

Author *

Start typing to see suggestions

ISBN-13

Publisher

Published Date

Description *

0 / 2000 characters

AI Assistant [Options](#)

Рисунок 4.2 – Форма додавання книги

Першим кроком користувач може ввести ISBN-код книги. Після введення коду можна надіслати запит до Google Books API, який заповнює всі доступні поля форми – назву, автора, видавництво, опис, категорії та зображення обкладинки (рис. 4.3).

Publisher: Enter publisher

Published Date: 1953

Description *: No description available

24 / 2000 characters

AI Assistant Options

Generate Description Improve Current

Page Count: 248

Language: English

Categories / Genres *: Select categories

Aerial photography x

Select at least one category

Cover Image

Remove

Рисунок 4.3 – Заповнена форма додавання книги

Якщо користувач не має ISBN-коду, він може заповнити форму вручну. При введенні назви книги спрацьовує debounced-пошук дублікатів по каталогу. У разі виявлення відображається попередження, представлення на рис. 4.4.

Submit a New Book

Can not find a book in our catalog? Submit it here and we will add it after review.

Similar books found in catalog



Steve Jobs
Walter Isaacson

Continue anyway

Quick Fill with ISBN

Enter an ISBN to automatically fill book details from Google Books

9781451648546

Lookup

Рисунок 4.4 – Знайдена існуюча книга

Для покращення якості опису книги користувач може натиснути кнопку «Згенерувати опис». Система збирає введені метадані та генерує літературну анотацію обсягом 150–200 слів, візуалізація показана на рис. 4.5.

Description *

Enter book description

0 / 2000 characters

AI Assistant Options

Generate Description

Ever looked at an old aerial photograph and wondered about the brilliant minds who first learned to truly *read* the landscape from above? This incredible collection takes you right back to the pivotal year of 1953, offering a unique window into the early days of photogeology and photo interpretation. It gathers selected papers from experts at the United States' own Research and Development Board, showcasing the groundbreaking techniques and profound insights they gleaned from aerial imagery. Think of it as a foundational text, revealing how pioneers identified geological formations, mapped geographical features, and understood complex terrains long before the digital age. If you're a history buff for science, fascinated by the evolution of remote sensing, or simply curious about the ingenious ways experts deciphered our planet from a bird's-eye view, you'll find this a truly compelling and enriching journey. It's a testament to the power of observation and early scientific rigor.

995 characters

Regenerate Apply

Рисунок 4.5 – Генерація опису книги

Після заповнення всіх обов'язкових полів та проходження валідації користувач натискає кнопку відправки. Книга отримує статус «на модерації» та очікує перевірки адміністратором. Дана сторінка представлена на рис 4.6.

My Book Submissions

Track the status of your book submissions

Submit New Book



Selected Papers on Photogeology and Photo Interpretation, Presented at Meetings Sponsored by the Committee on Geophysics and Geography, Research and Development Board

Author: United States, Research and Development Board, Committee on Geophysics and Geography

Submitted less than a minute ago

Pending

Ever looked at an old aerial photograph and wondered about the brilliant minds who first learned to truly *read* the landscape from above? This incredible collection takes you right back to the pivotal year of 1953, offering a unique window into the early days of photogeology and photo interpretation. It gathers selected papers from experts at the United States' own Research and Development Board, showcasing the groundbreaking techniques an...

Aerial photography



Migrant Mother, Migrant Gender: Reconsidering Dorothea Lange's Iconic Portrait of Maternity

Author: Sally Stein

Publisher: Mack

Approved

Рисунок 4.6 – Книжки відправлені на модерацію користувачем

Наступними розглянемо приклади роботи аналітики різних типів, тому спочатку маємо представлений алгоритм генерації аналітики на рис. 4.7.

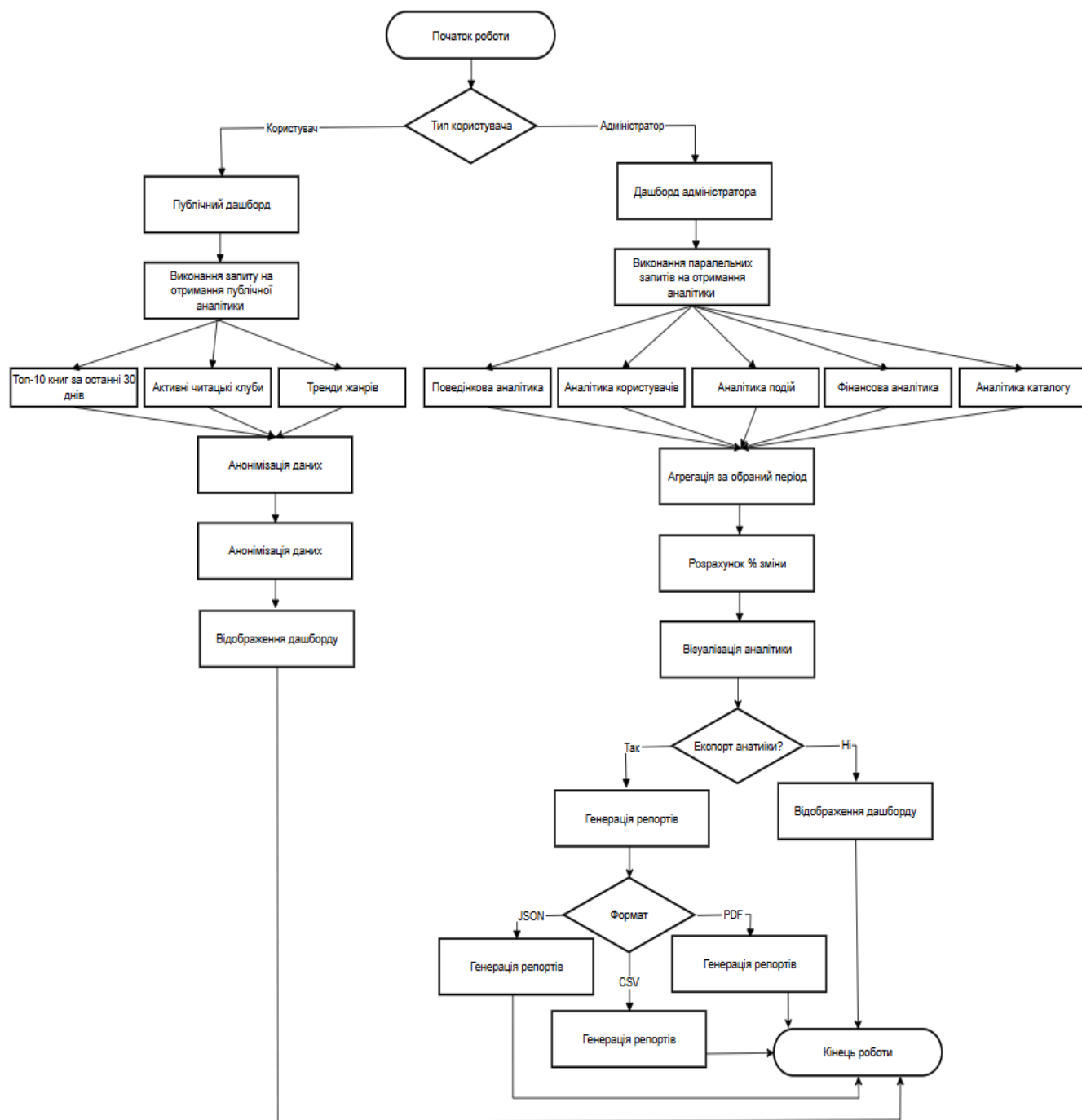


Рисунок 4.7 – Блок схема процесу генерації аналітики

Дашборд аналітики адміністратора відображає ключові метрики у вигляді карток: дохід, кількість замовлень, кількість користувачів, кількість книг.

Кожна картка на рис. 4.8 містить значення зміни порівняно з попереднім періодом:

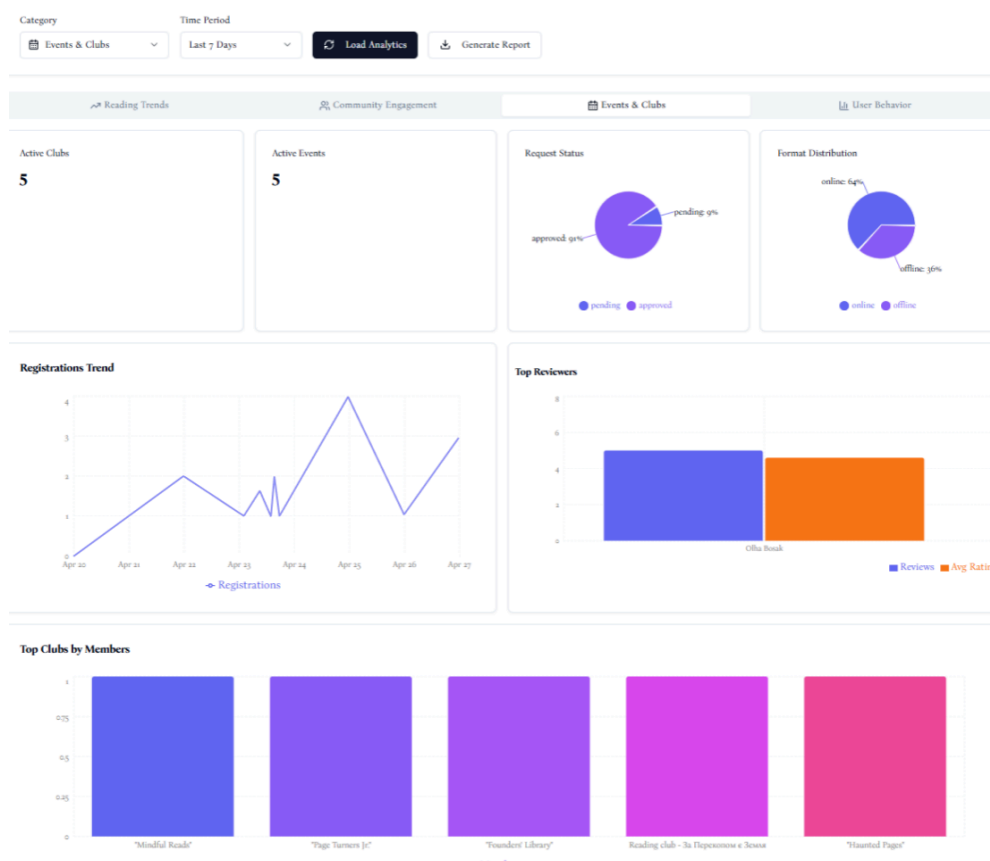


Рисунок 4.8 – Приклад результату генерації аналітики адміністратором

Для експорту звіту адміністратор натискає кнопку «Експортувати» та обирає формат – JSON, CSV або PDF. Вебзастосунок генерує комплексний звіт з усіма метриками за обраний період. При виборі PDF-формату генерується структурований документ із заголовками, таблицями та графіками через бібліотеку jsPDF[17].

Публічна сторінка аналітики на рис. 4.9 доступна всім відвідувачам і відображає загальну статистику платформи: топ-10 книг місяця, активні читацькі клуби та тренди жанрів. Дані оновлюються автоматично та дозволяють користувачам відстежувати популярні напрямки літератури, що сприяє залученню нової аудиторії та підвищенню інтересу до вебзастосунку.

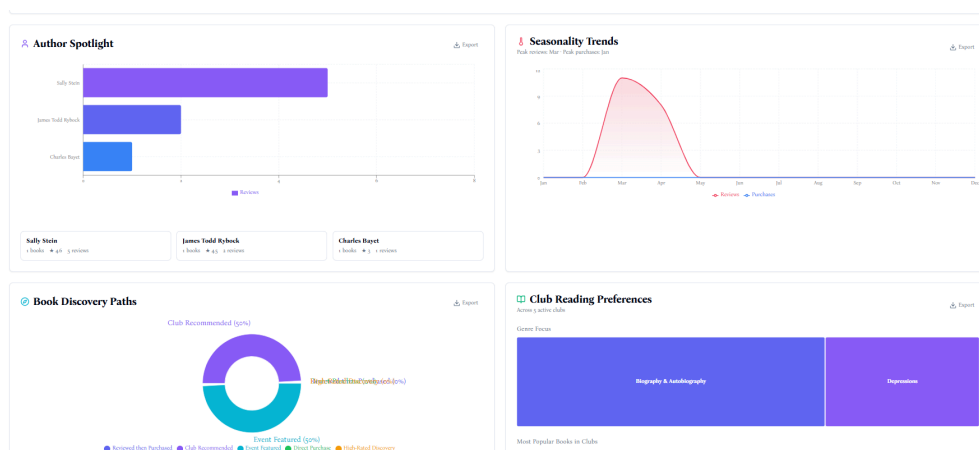


Рисунок 4.9 – Приклад публічної аналітики

Компонент Community Insights на рис. 4.10 відображає агреговану анонімізовану статистику спільноти – середню кількість книг на користувача та популярні часи для читацьких клубів.

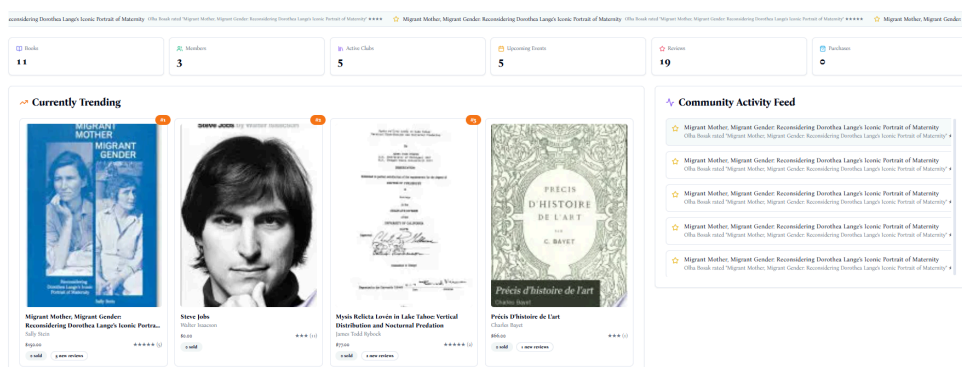


Рисунок 4.10 – Анонімізована статистика спільноти

Висновки до розділу 4

У даному розділі було детально розглянуто процес програмної реалізації вебзастосунку. Перш за все було обґрунтовано вибір інструментів розробки, проаналізовано доступні рішення та визначено стек технологій для реалізації поставлених завдань. Також було описано технологію створення вебзастосунку, включаючи процеси наповнення каталогу, систему аналітики. Представлено скріншоти інтерфейсу та описано ключові сценарії використання різними категоріями користувачів.

5 ТЕСТУВАННЯ ФУНКЦІОНАЛЬНОСТІ

5.1 Запровадження юніт тестування

Юніт-тестування спрямоване на перевірку коректності роботи окремих функцій у ізоляції від зовнішніх залежностей. Для реалізації юніт-тестів використано Jest[18] з React Testing Library[19]. Зовнішні сервіси, замінюються mock-об'єктами, що дозволяє тестувати виключно логіку конкретного модуля. У межах юніт-тестування було покрито такі модулі:

- валідація даних: тестуються схеми валідації форми додавання книги, форми відгуку, форми реєстрації та авторизації, перевіряється коректність валідації обов'язкових полів, обмежень довжини тексту;
- утилітарні функції: тестуються допоміжні функції форматування дати, обчислення середнього рейтингу, пагінації результатів, нормалізації пошукових запитів та перетворення даних Google Books API у внутрішній формат моделі;
- AI-сервіс (lib/ai.ts): тестується формування промптів для генерації опису книги, перевіряється, що функції коректно включають метадані книги, обраний тон, довжину та мову до тексту промпу;
- серверна валідація API-ендпоінтів: тестується логіка валідації вхідних параметрів route handlers – перевірка автентифікації, перевірка обов'язкових полів;
- логіка обчислення аналітики: тестуються функції агрегації статистичних даних, обчислення загальної кількості книг за статусами, обчислення середнього рейтингу та формування даних;
- допоміжні функції авторизації: тестується логіка перевірки ролей користувача – функції визначення прав доступу адміністратора;
- React-компоненти: за допомогою React Testing Library тестується рендеринг ключових компонентів інтерфейсу.

Загалом було створено 127 юніт-тестів, що охоплюють критичну бізнес-логіку платформи. Детальний звіт покриття коду юніт-тестами наведено у підрозділі 5.2 спільно з інтеграційними тестами, оскільки обидва рівні тестування

реалізовані за допомогою Jest та React Testing Library, і інструмент генерує єдиний агрегований звіт покриття.

5.2 Запровадження інтеграційного тестування

Інтеграційне тестування спрямоване на перевірку коректності взаємодії між кількома компонентами. На відміну від юніт-тестів, інтеграційні тести перевіряють реальну взаємодію між шарами – від API route handler через бізнес-логіку до бази даних. Тестуються такі процеси:

- цикл створення книги через API endpoint – від отримання HTTP-запиту, серверної валідації, перевірки дублікатів у базі даних до збереження запису зі статусом «pending». Перевіряється, що запис реально створюється в базі даних;
- схвалення та відхилення книги адміністратором - перевіряється зміна статусу з «pending» на «approved» або «rejected», перевірки прав доступу;
- створення відгуку з рейтингом, прив'язка відгуку до конкретної книги та користувача, обчислення середнього рейтингу книги після додавання відгуку;
- взаємодія route handler з AI-сервісом – перевіряється коректне формування запиту до Google Gemini API, обробка streaming-відповіді, обробка таймауту.
- генерація аналітичних звітів – перевіряється коректність запитів при різних обсягах даних, правильність фільтрації за часовими періодами та відповідність повернених статистичних показників фактичному стану бази даних.
- цикл перевірки сесії через NextAuth – від отримання токenu до визначення ролі користувача та надання або відмови у доступі до захищених ендпоінтів.

Оскільки юніт-тести та інтеграційні тести реалізовані за допомогою єдиного стеку інструментів, звіт покриття коду генерується спільно для обох рівнів тестування. Jest Coverage Report агрегує дані покриття з усіх тестових файлів незалежно від того, чи тест перевіряє окрему функцію в ізоляції, чи взаємодію кількох компонентів. Загальне покриття кодової бази юніт та інтеграційними тестами становить 87%.

На рис. 5.1 представлено зведений звіт покриття коду.

Full 11185
93.26% Statements (3348/3592) 88.94% Branches (3529/3980) 89.72% Functions (449/500) 93.26% Lines (3348/3592)

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter

File	Statements	Branches	Functions	Lines
src/features/auth/components/forms/ResetPasswordForm	100%	44/44	100%	6/6
src/features/auth/components/forms/SignInForm	100%	69/69	87.5%	7/8
src/features/auth/components/forms/SignUpForm	100%	48/48	66.66%	2/3
src/features/auth/components/ui/AuthPageLayout	100%	84/84	100%	1/1
src/features/auth/components/ui/ReviewsSlider	100%	126/126	100%	6/6
src/features/auth/hooks	99.01%	403/407	86.04%	37/43
src/features/auth/lib	100%	134/134	96%	24/25
src/features/auth/utils	100%	63/63	100%	4/4
src/features/catalog/api	100%	103/103	100%	6/6
src/features/catalog/components/ui/AccordionIcon	100%	12/12	100%	1/1
src/features/catalog/components/ui/Catalog	100%	244/244	94.44%	17/18
src/features/catalog/components/ui/CheckboxList	100%	68/68	100%	3/3
src/features/catalog/components/ui/FilterAccordion	100%	42/42	100%	1/1
src/features/catalog/components/ui/FilterList	100%	247/247	97.43%	38/39
src/features/catalog/components/ui/ProductsList	100%	110/110	100%	8/8
src/features/catalog/hooks	97.5%	117/120	88%	22/25
src/features/internal-server-error/hooks	100%	39/39	85.71%	6/7
src/features/my-wishlist/components/AProductRecommendations	100%	104/104	100%	14/14
src/features/my-wishlist/components/WishlistDeleteButton	100%	42/42	100%	5/5
src/features/my-wishlist/hooks	99.34%	304/306	96.77%	60/62
src/features/not-found/hooks	100%	37/37	80%	4/5
src/features/order-history/components/DeliveryInfo	100%	37/37	100%	2/2
src/features/order-history/components/DownloadInfo	90.74%	49/54	44.44%	4/9
src/features/order-history/components/OrderCard	100%	46/46	100%	5/5
src/features/order-history/components/OrderCardHeader	100%	59/59	100%	1/1
src/features/order-history/components/OrderCardFooter	100%	73/73	100%	3/3

Рисунок 5.1 – Звіт покриття коду юніт та інтеграційними тестами

Сумарно на двох рівнях тестування було створено 195 тестів (127 юніт + 68 інтеграційних), з яких 195 завершуються успішно.

5.3 Запровадження end-to-end тестування

End-to-end тестування спрямоване на перевірку повних користувацьких сценаріїв у реальному браузерному середовищі. Тести виконуються за допомогою Cypress[20], який запускає браузер, взаємодіє з інтерфейсом так, як це робив би користувач, і перевіряє коректність відображення результатів. Було протестовано основні користувацькі сценарії платформи: реєстрація нового користувача та авторизація, перегляд каталогу книг з пошуком та фільтрацією, перегляд деталей книги та додавання відгуку з рейтингом, формування каталогу, модерація книги, генерація аналітики адміністратором. Сценарій формування каталогу (додавання книги користувачем та адміністратором) є одним із найкомплексніших, оскільки охоплює взаємодію з зовнішніми сервісами, багатоетапну валідацію та різну поведінку залежно від ролі користувача. End-to-end тест цього сценарію

складається з двох підсценаріїв: підсценарій 1: додавання книги звичайним користувачем. Підсценарій 2: додавання книги адміністратором. На рис. 5.2 представлений скріншот із середовища Cypress.

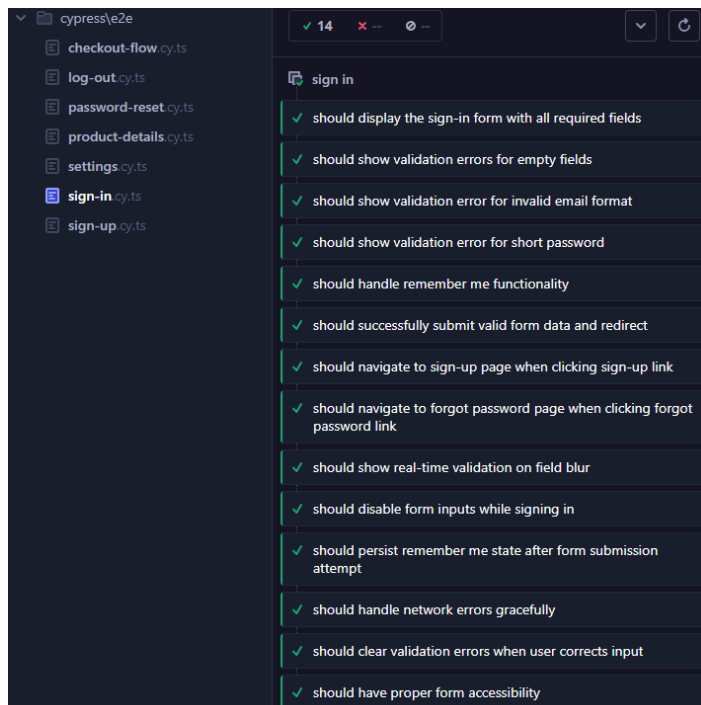


Рисунок 5.2 – Виконання end-to-end тестування

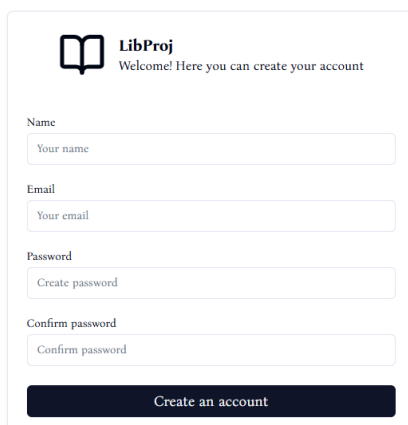
Висновки до розділу 5

У межах розділу було запроваджено стратегію тестування що охоплює юніт-тести, інтеграційні тести та end-to-end тести. На рівні юніт-тестування було створено 127 тестів, що перевіряють роботу окремих модулів в ізоляції. На рівні інтеграційного тестування було створено 68 тестів, що перевіряють взаємодію, від API route handlers через бізнес-логіку до тестової бази даних PostgreSQL. Зведене покриття кодової бази юніт та інтеграційними тестами, реалізованими за допомогою єдиного стеку Jest та React Testing Library, становить 87%. На рівні end-to-end тестування було створено 23 тести, що охоплюють основні користувацькі сценарії у реальному браузерному середовищі. Загалом створено 218 тестів, що забезпечують покриття функціональності платформи та дозволяють виявляти регресії на ранніх етапах розробки.

6 ЕКСПЛУАТАЦІЯ ВЕБЗАСТОСУНКУ

6.1 Інструкція користувача

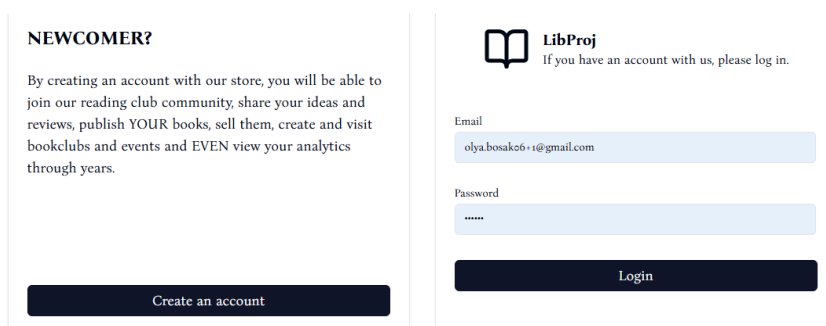
Вебзастосунок розгорнутий на платформі Vercel та доступний через будь-який сучасний браузер. Для створення облікового запису користувач натискає кнопку «Зареєструватись». Відображається форма реєстрації з полями: ім'я, електронна пошта та пароль. Після заповнення полів відбувається валідація даних, створюється обліковий запис. На рис. 6.1 представлено форму реєстрації.



The registration form for LibProj is displayed within a light gray border. At the top left is a book icon, followed by the text 'LibProj' and 'Welcome! Here you can create your account'. Below this are four input fields: 'Name' with placeholder 'Your name', 'Email' with placeholder 'Your email', 'Password' with placeholder 'Create password', and 'Confirm password' with placeholder 'Confirm password'. At the bottom is a dark blue button with the text 'Create an account'.

Рисунок 6.1 – Форма реєстрації користувача

Для входу в систему користувач натискає кнопку «Увійти» та вводить електронну пошту і пароль. Після успішної перевірки облікових даних вебзастосунок створює JWT-сесію. У разі введення невірних даних відображається повідомлення. На рис. 6.2 представлено форму авторизації.



The login and newcomer form for LibProj is displayed within a light gray border. On the left side, under the heading 'NEWCOMER?', there is a paragraph of text: 'By creating an account with our store, you will be able to join our reading club community, share your ideas and reviews, publish YOUR books, sell them, create and visit bookclubs and events and EVEN view your analytics through years.' Below this text is a dark blue button with the text 'Create an account'. On the right side, there is a book icon, followed by the text 'LibProj' and 'If you have an account with us, please log in.' Below this are two input fields: 'Email' with placeholder 'olya.bosakc6+1@gmail.com' and 'Password' with placeholder '.....'. At the bottom is a dark blue button with the text 'Login'.

Рисунок 6.2 – Форма авторизації користувача

Каталог мистецької літератури доступний у вигляді карток із назвою, автором, рейтингом та ціною. Доступна розширена фільтрація та сортування. На рис. 6.3 представлено сторінку каталогу з результатами пошуку.

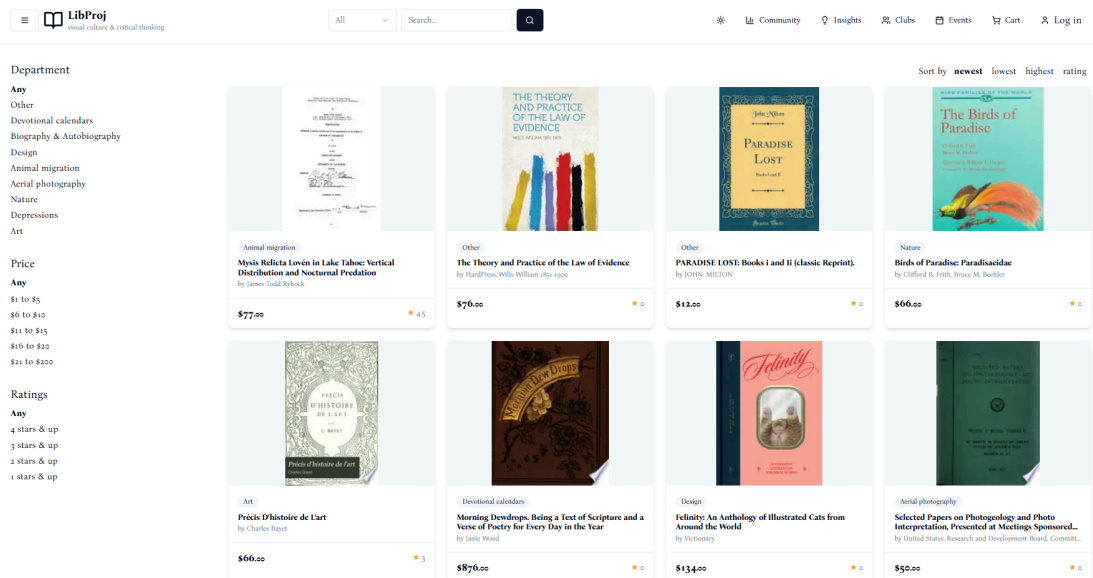


Рисунок 6.3 – Сторінка каталогу книг із фільтрацією та пошуком

На сторінці книги користувач натискає кнопку «Додати в кошик». Якщо книга доступна до продажу, то вона додається в кошик. У разі якщо книга вже є в кошику, збільшується її кількість. На рис. 6.4 представлено сторінку кошика з обраними книгами.

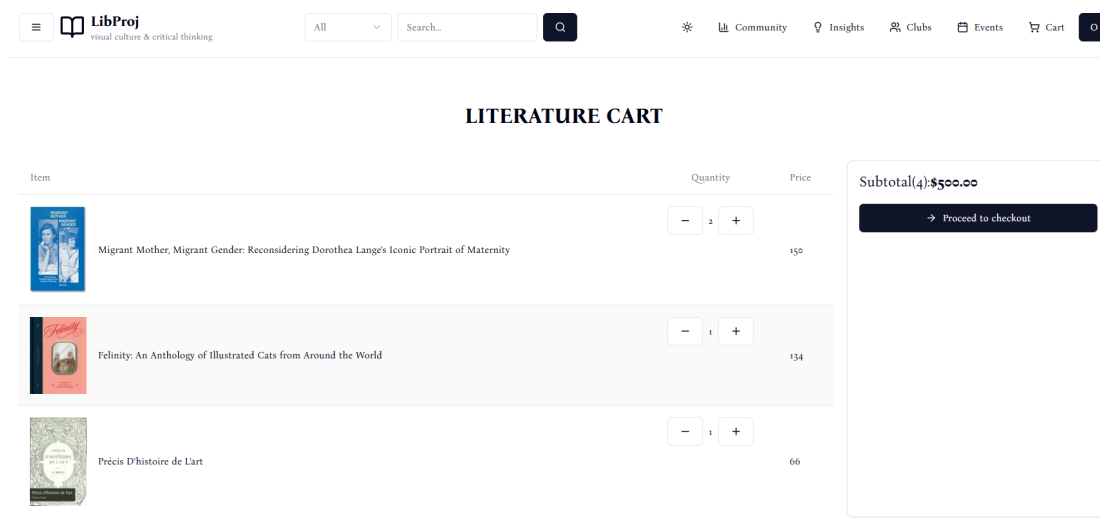


Рисунок 6.4 – Сторінка кошику покупок із обраними книгами

Після формування кошика користувач натискає кнопку «Перейти до оформлення». Відображається сторінка оформлення замовлення, де необхідно ввести особисті дані. Далі користувач переходить до етапу оплати через PayPal. На рис. 6.5 представлено останню сторінку оформлення замовлення.

Order .oe2414

Payment Method
PayPal
Not paid

Shipping Address
Olha Bosak
Mykhailiska 8, Odesa 65005, Ukraine
Not Delivered

Order Items

Item	Quantity	Price
Migrant Mother, Migrant Gender: Reconsidering Dorothea Lange's Iconic Portrait of Maternity	2	\$150
Felinity: An Anthology of Illustrated Cats from Around the World	1	\$134

Items \$500.00
Tax \$75.00
Shipping \$0.00
Total \$575.00

PayPal
Debit or Credit Card
Powered by PayPal

Рисунок 6.5 – Сторінка оформлення замовлення

Авторизований користувач може створити літературну подію або читацький клуб через розділ «Події/Клуби». Форма створення включає поля: назва, опис, дата та час проведення, місце, книга для обговорення, максимальна кількість учасників. На рис. 6.6 представлено форму створення літературної події або клубу.

Create a Reading Club or Event

Start a reading club for ongoing book discussions or create a one-time event. Your request will be reviewed by our team before it goes live.

Type
☒ Reading Club ☐ Event
A reading club is an ongoing group that meets regularly

Title
Enter a catchy title

Start Date
mm/dd/yyyy -- --

End Date (Optional)
mm/dd/yyyy -- --
Leave empty for ongoing clubs

Capacity
10
Maximum number of participants

Number of Sessions
4
Total meetings planned

Format
☒ Online ☐ Offline

Online Meeting Link
https://meet.example.com/your-meeting
Zoom, Google Meet, or any other platform link

Select Books
Choose one or more books to read and discuss

☐ Mysis Relicta Lovén in Lake Tahoe: Vertical Distribution and Nocturnal Predation by James Todd Rybock

☐ The Theory and Practice of the Law of Evidence by HardPress, Willis William 1899

☐ PARADISE LOST: Books i and II (classic Reprint). by JOHN MILTON

Рисунок 6.6 – Форма створення літературної події або клубу

На сторінці книги авторизований користувач може залишити рецензію у відповідному розділі та переглянути AI-резюме з визначенням тональності та аспектів усіх відгуків. На рис. 6.7 представлено сторінку залишення рецензії.

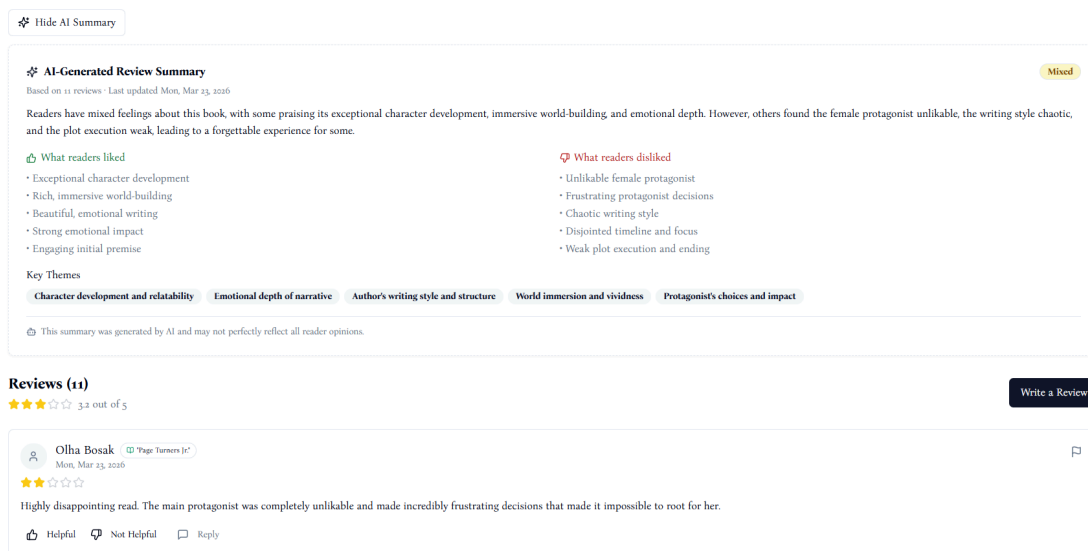


Рисунок 6.7 – Сторінка написання рецензії

У розділах подій та клубів відображаються списки доступних із описом, датою, книгою для обговорення та кількістю учасників. Для приєднання користувач натискає кнопку «Приєднатись» на сторінці обраної події або клубу. На рис. 6.8 представлено сторінку читацького клубу з можливістю приєднання.

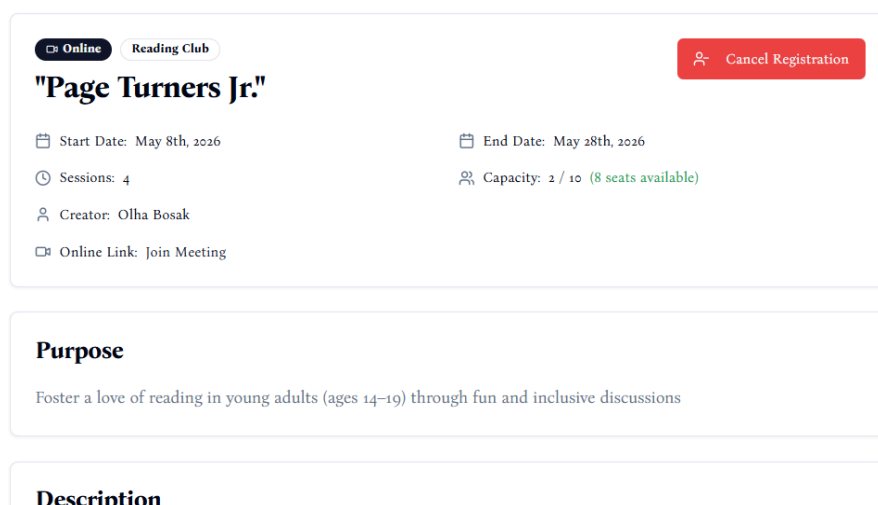


Рисунок 6.8 – Сторінка читацького клубу з кнопкою приєднання

Адміністратор має доступ до повного управління каталогом. Для кожної книги доступні дії: перегляд деталей, редагування метаданих та видалення з каталогу. Редагування відбувається через ту саму форму, що й створення, з попередньо заповненими полями. На рис. 6.9 представлено сторінку управління.

LibProj visual culture & critical thinking						
Overview	Literature	Orders	Users	Moderate Requests	Book Submissions	Analytics
Search...						
Community Insights Clubs Events Cart						
Literature						
ID	NAME	PRICE	CATEGORY	STOCK	RATING	ACTIONS
.7a6e16	Mysis Relicta Lovén in Lake Tahoe: Vertical Distribution and Nocturnal Predation	\$77.00	Animal migration	2	4.5	Edit Delete
.7b54fb	The Theory and Practice of the Law of Evidence	\$76.00	Other	5	0	Edit Delete
.a19f4b	PARADISE LOST: Books i and ii (classic Reprint).	\$12.00	Other	1	0	Edit Delete
.3c435d	Birds of Paradise: Paradisacidae	\$66.00	Nature	66	0	Edit Delete
.c2b4c0	Précis D'histoire de l'art	\$66.00	Art	77	3	Edit Delete
.a2bbab	Morning Dewdrops. Being a Text of Scripture and a Verse of Poetry for Every Day in the Year	\$876.00	Devotional calendars	33	0	Edit Delete
.9bdd43	Felinity: An Anthology of Illustrated Cats from Around the World	\$134.00	Design	55	0	Edit Delete
.b1e656	Selected Papers on Photogeology and Photo Interpretation, Presented at Meetings Sponsored by the Committee on Geophysics and Geography, Research and Development Board	\$50.00	Aerial photography	12	4.5	Edit Delete
.c6edcb	Migrant Mother, Migrant Gender: Reconsidering Dorothea Lange's Iconic Portrait of Maternity	\$150.00	Depressions	10	4.6	Edit Delete

Рисунок 6.9 – Сторінка панелі адміністратора для управління каталогом

Також маємо схожий розділ, який відображає таблицю всіх замовлень із інформацією: номер, дата, користувач, сума, статус. Адміністратор може переглянути деталі кожного замовлення, оновити його статус або скасувати. На рис. 6.10 представлено сторінку управління замовленнями.

LibProj visual culture & critical thinking					
Overview	Literature	Orders	Users	Moderate Requests	Book Submissions
Analytics Search...					
Community Insights Clubs					
Orders					
ID	DATE	TOTAL	PAID	DELIVERED	ACTIONS
.0e2414	Apr 20, 2026, 3:44 PM	\$575.00	Not Paid	Not Delivered	Details Delete
.0e2414	Apr 20, 2026, 3:44 PM	\$575.00	Not Paid	Not Delivered	Details Delete
.0e2414	Apr 20, 2026, 3:44 PM	\$575.00	Not Paid	Not Delivered	Details Delete
.0e2414	Apr 20, 2026, 3:44 PM	\$575.00	Not Paid	Not Delivered	Details Delete
.0e2414	Apr 20, 2026, 3:44 PM	\$575.00	Not Paid	Not Delivered	Details Delete
.0e2414	Apr 20, 2026, 3:44 PM	\$575.00	Not Paid	Not Delivered	Details Delete
.0e2414	Apr 20, 2026, 3:44 PM	\$575.00	Not Paid	Not Delivered	Details Delete
.0e2414	Apr 20, 2026, 3:44 PM	\$575.00	Not Paid	Not Delivered	Details Delete
.0e2414	Apr 20, 2026, 3:44 PM	\$575.00	Not Paid	Not Delivered	Details Delete

Рисунок 6.10 – Сторінка панелі адміністратора для управління замовленнями

Створені події та клуби потрапляють на модерацию. У відповідних розділах відображаються списки подій та клубів зі статусом «на модерації». Адміністратор переглядає деталі, перевіряє відповідність правилам платформи та натискає схвалення або відхилення. На рис. 6.11 представлено сторінку модерації клубів.

Book Swap & Brew: Spring Exchange Meetup Event

Build community connections by exchanging pre-loved books over coffee in a casual, welcoming environment

Submitted By
 Admin
 olya.bosak26-i@gmail.com
 April 29th, 2026

Start Date
 Jul 2, 2026

Capacity
 20

Format
 Online

Description
 Bring 1-3 books you've finished and leave with something new! Our Spring Book Swap & Brew is a relaxed social event where readers meet, mingle, and trade books. Each participant gives a 60-second 'pitch' for the books they're offering. We'll have coffee, tea, and pastries provided. All genres welcome. At the end of the event, unclaimed books will be donated to the local library. Come for the books, stay for the conversation!

Online Meeting Link
<https://meet.google.com/abc-defg-hij>

Selected Books
 1 book(s) selected for this event

Approve **Reject**

Рисунок 6.11 – Форма для модерації читачських клубів і подій адміністратором

Книги, подані користувачами також потрапляють до розділу модерації. Адміністратор переглядає метадані поданої книги: назву, автора, ISBN, опис, категорії. Після перевірки адміністратор може схвалити подання, або відхилити. На рис. 6.12 представлено інтерфейс модерації поданих книг.

Total Pending 1 Awaiting moderation

On Time 1 Within 3 day SLA

Recent 1 Last 24 hours

Overdue 0 >3 days old

High on Horses On Time
 Author: Nicole Emanuel, Nicole Clare Emanuel
 Submitted by: Admin (olya.bosak26-i@gmail.com)
 Submitted: 31 minutes ago

ISBN
 1804700149

ISBN-13
 9781804700145

Publisher
 Images Publishing

Published Date
 2003

Pages
 184

Language
 en

For Sale
 No

Description
 A feast of pictures displaying the incredible caring relationship that can exist between horse and rider.

Categories
 Art

Preview
 View on Google Books

Approve **Merge with Existing** **Reject**

Рисунок 6.12 – Сторінка для модерації книг, поданих користувачами

6.2 Опис експериментів та аналіз результатів

У межах роботи було виконано поставлені задачі:

- 1) проведено аналіз предметної області та існуючих рішень;
- 2) здійснено планування процесу розробки;
- 3) спроектовано архітектуру, структуру бази даних та програмні класи;
- 4) виконано програмну реалізацію;
- 5) проведено тестування та розгортання.

Оскільки метою роботи є підвищення рівня залученості користувачів до читання мистецької літератури та участі в читацьких клубах, експеримент спрямовано на кількісне вимірювання показників залученості при використанні вебзастосунку LibProj у порівнянні з аналогом – Goodreads.

Експеримент проводився за схемою «аналог - власна розробка»: група користувачів спочатку виконувала набір типових задач на платформі Goodreads протягом 20-хвилинної сесії, а потім – аналогічні задачі на розгорнутому екземплярі LibProj з тестовою базою даних (50 книжкових позицій, 10 читацьких клубів, 15 подій, 200 відгуків).

Під час кожної сесії фіксувалися кількісні показники активності та час виконання операцій. Результати усереднювалися по групі. Перелік задач, які виконували учасники:

- 1) знайти книгу за жанром «українська проза» та переглянути її сторінку;
- 2) знайти читацький клуб за тематикою та приєднатися до нього;
- 3) знайти літературну подію та зареєструватися на неї;
- 4) переглянути аналітику власної читацької активності;
- 5) ознайомитися з узагальненням відгуків на книгу.

Для кількісної оцінки залученості було обрано метрики, що безпосередньо відображають рівень взаємодії користувача з вебзастосунком та ефективність залучення до читацької діяльності (табл. 6.1).

Таблиця 6.1 – Кількісні показники залученості користувачів

Метрика	Goodreads	LibProj
Середній час пошуку книги	45 с	18 с
Середня кількість переглянутих книг за сесію	4,2	8,6
Кількість знайдених літературних подій	0 (Не реалізовано)	6,4
Кількість приєднань до клубів	0,8	2,4
Кількість реєстрацій на події	0 (Не реалізовано)	1,8
Доступні метрики аналітики читання	1 (річний лічильник)	7
AI-узагальнення відгуків	Ні	Так
Середній час отримання враження з відгуків	120 с (ручний перегляд)	5 с (AI-резюме)
Середня кількість дій (кліків)	12,3	24,7
Кількість приєднань до клубів	0,8	2,4
Кількість реєстрацій на події	0 (Не реалізовано)	1,8

Експеримент виявив підвищення рівня залученості користувачів при використанні LibProj порівняно з Goodreads за всіма вимірюваними метриками. За критерієм ефективності пошуку середній час знаходження книги скоротився у 2,5 рази (з 45 с до 18 с) завдяки інтегрованій системі фільтрації з орієнтацією на мистецьку літературу. Це призвело до зростання кількості переглянутих книг за сесію вдвічі (з 4,2 до 8,6), що свідчить про підвищення інтересу до перегляду каталогу. За критерієм участі в спільній діяльності кількість приєднань до читацьких клубів зросла втричі (з 0,8 до 2,4 за сесію), а час приєднання скоротився вдвічі (з 70 с до 35 с). Реєстрації на літературні події (в середньому 1,8 за сесію) стали можливими виключно завдяки LibProj, оскільки Goodreads не має цього функціоналу. Час отримання загального враження про книгу скоротився у 24 рази (з 120 с до 5 с) завдяки AI-узагальненню відгуків. Загальна активність взаємодії (кількість дій за сесію) зросла вдвічі (з 12,3 до 24,7), що є інтегральним показником підвищення залученості. Представимо результати у вигляді радарної

діаграми, яка призначена для порівняння характеристик обраних об'єктів. На рис.6.13 представлена дана діаграма.

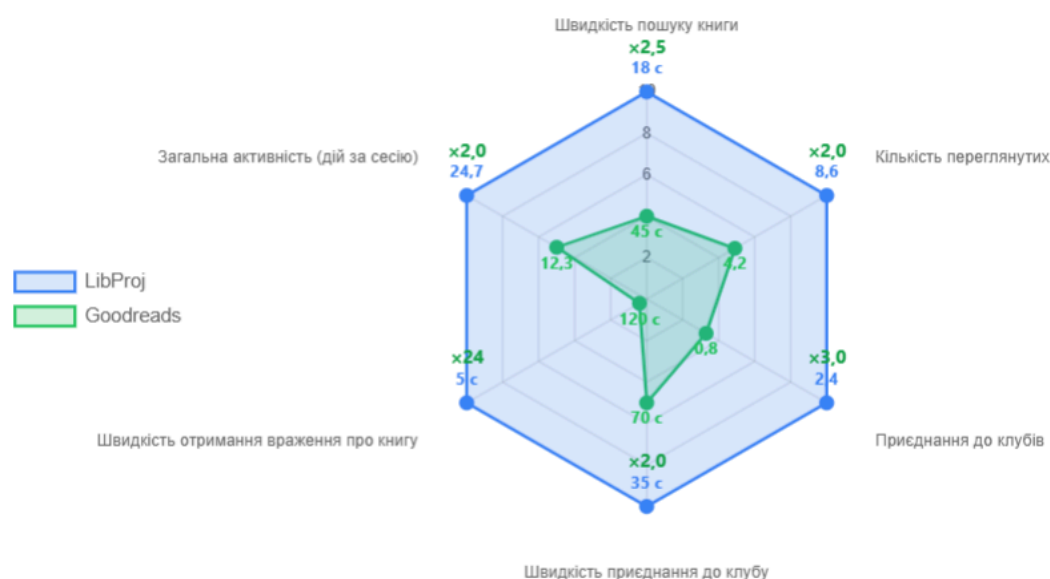


Рисунок 6.13 – Радарна діаграма порівняння рівня залученості користувачів

Мету роботи досягнуто – розроблений вебзастосунок LibProj кількісно підтверджує підвищення рівня залученості користувачів до читання мистецької літератури та участі в читацьких клубах: швидкість пошуку літератури зросла у 2,5 рази, кількість переглянутих книг – у 2 рази, приєднань до клубів – у 3 рази, а загальна активність взаємодії – у 2 рази.

Висновки до розділу 6

У межах розділу описано процес експлуатації розробленого вебзастосунку та надано інструкцію користувача, що охоплює основні сценарії взаємодії. Для підтвердження досягнення мети роботи проведено порівняльний експеримент із найближчим аналогом. Експеримент засвідчив суттєве підвищення рівня залученості користувачів при використанні LibProj: швидкість пошуку літератури зросла у 2,5 рази, кількість переглянутих книг за сесію – у 2 рази, приєднань до читацьких клубів – у 3 рази, а загальна активність – у 2 рази.

ЗАГАЛЬНІ ВИСНОВКИ

У кваліфікаційній роботі розроблено вебзастосунок для популяризації мистецької літератури та організації читацьких клубів. Поставлена мета – підвищення рівня залученості користувачів до читання літератури та участі в читацьких клубах – досягнута шляхом реалізації інструментів каталогу та аналітики читацьких уподобань.

У межах кваліфікаційної роботи було виконано поставлені задачі:

Проведено аналіз предметної області та існуючих рішень, виявлено відсутність єдиної платформи, що поєднує каталог літератури, читацькі клуби та аналітику. Сформульовано функціональні та нефункціональні вимоги.

Здійснено планування процесу розробки з оцінкою тривалості за методом UCP з використанням діаграми Ганта та аналізом потенційних ризиків із визначенням стратегій мінімізації.

Спроектовано модульну трирівневу архітектуру на базі Next.js App Router із використанням Server Actions, розроблено структуру бази даних у Prisma ORM, описано функціональні модулі вебзастосунку.

Виконано програмну реалізацію та описано роботу основних модулів: каталог мистецької літератури із фільтрацією, модуль подання книг з автозаповненням через Google Books API та AI-генерацією описів, інструменти створення читацьких клубів та літературних подій, оформлення замовлення, систему рецензій з оновленням рейтингів, аналітичний дашборд з експортом.

Проведено тестування на трьох рівнях (юніт, інтеграційне, end-to-end) та експлуатаційні випробування, що підтвердили коректність роботи інструментів.

Практичне значення отриманих результатів полягає у створенні готового до використання вебзастосунку, який може бути застосований літературними організаціями для масштабування існуючих ініціатив популяризації читання. Можливі напрями продовження роботи: інтеграція системи рекомендацій, реалізація мобільного застосунку, розширення аналітики за рахунок машинного навчання для прогнозування читацьких уподобань.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Goodreads Platform. URL: <https://www.goodreads.com> (дата звернення: 12.01.2026).
2. LibraryThing Platform. URL: <https://www.librarything.com> (дата звернення: 12.01.2026).
3. StoryGraph Platform. URL: <https://www.thestorygraph.com> (дата звернення: 12.01.2026).
4. Bookclubs Platform. URL: <https://bookclubs.com> (дата звернення: 12.01.2026).
5. Clemmons R. Project Estimation With Use Case Points. CrossTalk : The Journal of Defense Software Engineering 2006. URL: https://www.researchgate.net/publication/200036324_Project_Estimation_With_Use_Case_Points (дата звернення: 20.01.2026).
6. Діаграма послідовності. Цілі та призначення для побудови діаграм послідовності. URL: <https://e5.ua/uk/blogpost-2/sequence-diagram-dosi-zhiva/> (дата звернення: 26.02.2026).
7. Документація Next.js App Router. URL: <https://nextjs.org/docs> (дата звернення: 15.02.2026).
8. Документація сучасний TypeScript та патерни проєктування. URL: <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html> (дата звернення: 18.02.2026).
9. Prisma ORM Documentation. URL: <https://www.prisma.io/docs> (дата звернення: 18.02.2026).
10. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/current/> (дата звернення: 18.02.2026).
11. NextAuth.js Documentation. URL: <https://next-auth.js.org> (дата звернення: 20.02.2026).

12. Vercel AI SDK Documentation. URL: <https://sdk.vercel.ai/docs> (дата звернення: 20.02.2026).
13. PayPal Developer Documentation. URL: <https://developer.paypal.com> (дата звернення: 20.02.2026).
14. Google Books API Documentation. URL: <https://developers.google.com/books> (дата звернення: 20.02.2026).
15. UploadThing. File Uploads For Developers. URL: <https://docs.uploadthing.com/getting-started/appdir> (дата звернення: 20.02.2026).
16. Zod Schema Validation Documentation. URL: <https://zod.dev> (дата звернення: 25.02.2026).
17. The leading HTML5 client solution for generating PDFs. URL: <https://parall.ax/products/jspdf> (дата звернення: 15.03.2026).
18. Jest Testing Framework Documentation. URL: <https://jestjs.io> (дата звернення: 25.03.2026).
19. React Testing Library Documentation. URL: <https://testing-library.com/docs/> (дата звернення: 25.03.2026).
20. Cypress End-to-End Testing Documentation. URL: <https://docs.cypress.io/guides/overview/why-cypress> (дата звернення: 25.03.2026).

ДОДАТОК А

Лістинг програми

```
// Zod schema for book submission validation

export const bookSubmissionSchema = z.object({
  title: z
    .string()
    .min(1, 'Title must be at least 1 character')
    .max(255, 'Title must be at most 255 characters'),
  author: z
    .string()
    .min(1, 'Author must be at least 1 character')
    .max(128, 'Author must be at most 128 characters'),
  isbn: z
    .string()
    .optional()
    .nullable()
    .refine(
      (val) => {
        if (!val) return true;
        const cleaned = val.replace(/[-\s]/g, '');
        return /^(\d{10}|\d{13})$/.test(cleaned);
      },
      { message: 'ISBN must be 10 or 13 digits' },
    ),
  isbn13: z
    .string()
    .optional()
    .nullable()
    .refine(
      (val) => {
        if (!val) return true;
        const cleaned = val.replace(/[-\s]/g, '');
        return /^\d{13}$/.test(cleaned);
      },
      { message: 'ISBN-13 must be 13 digits' },
    ),
  publisher: z
    .string()
    .max(128, 'Publisher must be at most 128 characters')
    .optional()
    .nullable(),
  publishedDate: z
    .string()
    .max(64, 'Published date must be at most 64 characters')
    .optional()
    .nullable(),
  description: z
    .string()
    .min(10, 'Description must be at least 10 characters')
    .max(2000, 'Description must be at most 2000 characters'),
  pageCount: z.coerce
    .number()
    .int()
    .min(1, 'Page count must be at least 1')
    .max(10000, 'Page count must be at most 10000')
    .optional()
    .nullable(),
  language: z
    .string()
    .max(16, 'Language must be at most 16 characters')
```

```

        .optional()
        .nullable(),
categories: z.array(z.string()).min(1, 'At least one category is required'),
coverImage: z
    .string()
    .url('Cover image must be a valid URL')
    .optional()
    .nullable(),
thumbnailImage: z.string().optional().nullable(),
previewLink: z.string().url('Preview link must be a valid
URL').optional().nullable(),
googleBooksId: z.string().optional().nullable(),
isForSale: z.boolean().default(false),
suggestedPrice: z.coerce
    .number()
    .min(0.01, 'Price must be at least 0.01')
    .max(99999, 'Price must be at most 99999')
    .optional()
    .nullable(),
}).refine(
    (data) => {
        if (data.isForSale && (!data.suggestedPrice || data.suggestedPrice <= 0)) {
            return false;
        }
        return true;
    },
    {
        message: 'Price is required when the book is for sale',
        path: ['suggestedPrice'],
    },
);

```

// Normalization of the Google Books API response

```

function parseGoogleBookItem(item: GoogleBookItem): ParsedBookData {
    const volumeInfo = item.volumeInfo;

    let isbn: string | undefined;
    let isbn13: string | undefined;

    if (volumeInfo.industryIdentifiers) {
        for (const identifier of volumeInfo.industryIdentifiers) {
            if (identifier.type === 'ISBN_10') {
                isbn = identifier.identifier;
            } else if (identifier.type === 'ISBN_13') {
                isbn13 = identifier.identifier;
            }
        }
    }

    let coverImage: string | undefined;
    let thumbnailImage: string | undefined;

    if (volumeInfo.imageLinks) {
        coverImage =
            volumeInfo.imageLinks.extraLarge ||
            volumeInfo.imageLinks.large ||
            volumeInfo.imageLinks.medium ||
            volumeInfo.imageLinks.small ||
            volumeInfo.imageLinks.thumbnail;

        thumbnailImage =
            volumeInfo.imageLinks.smallThumbnail || volumeInfo.imageLinks.thumbnail;
    }
}

```

```

        if (coverImage) coverImage = coverImage.replace('http://', 'https://');
        if (thumbnailImage)
            thumbnailImage = thumbnailImage.replace('http://', 'https://');
    }

    return {
        title: volumeInfo.title + (volumeInfo.subtitle ? `: ${volumeInfo.subtitle}` :
    ''),
        author: volumeInfo.authors ? volumeInfo.authors.join(', ') : 'Unknown Author',
        isbn,
        isbn13,
        publisher: volumeInfo.publisher,
        publishedDate: volumeInfo.publishedDate,
        description: volumeInfo.description || 'No description available',
        pageCount: volumeInfo.pageCount,
        language: volumeInfo.language,
        categories: volumeInfo.categories || [],
        coverImage,
        thumbnailImage,
        previewLink: volumeInfo.previewLink,
        googleBooksId: item.id,
    };
}

// Generation of the book description via AI model with streaming response

import { google } from '@ai-sdk/google';
import { streamText } from 'ai';
import { BOOK_LANGUAGES } from './constants';

export type DescriptionTone = 'academic' | 'casual' | 'promotional';
export type DescriptionLength = 'short' | 'medium' | 'long';
export type GenerateMode = 'generate' | 'improve';

export interface BookContext {
    title: string;
    author: string;
    categories: string[];
    language?: string;
    publishedDate?: string;
    pageCount?: number | null;
    publisher?: string;
    existingDescription?: string;
}

export interface GenerateDescriptionParams {
    book: BookContext;
    mode: GenerateMode;
    tone: DescriptionTone;
    length: DescriptionLength;
    targetLanguage?: string;
}

const LENGTH_GUIDE: Record<DescriptionLength, string> = {
    short: '2-3 sentences (around 50-80 words)',
    medium: '1-2 paragraphs (around 120-200 words)',
    long: '2-3 paragraphs (around 250-400 words)',
};

const TONE_GUIDE: Record<DescriptionTone, string> = {
    academic:
        'Use a formal, scholarly tone. Focus on the work\'s thematic depth, literary
    merit, and contribution to its field or genre. Avoid sensationalism.',
    casual:

```

```

    'Use a warm, conversational tone as if recommending the book to a friend. Be
    engaging and approachable while still informative.',
    promotional:
    'Use compelling, marketing-oriented language. Highlight what makes this book a
    must-read. Create urgency and excitement without being misleading.',
  };

function getLanguageName(code?: string): string {
  if (!code) return 'English';
  const found = BOOK_LANGUAGES.find((l) => l.code === code);
  return found?.name || 'English';
}

function buildPrompt(params: GenerateDescriptionParams): string {
  const { book, mode, tone, length, targetLanguage } = params;
  const lang = getLanguageName(targetLanguage || book.language);

  const metadata = [
    `Title: "${book.title}"`,
    `Author: ${book.author}`,
    book.categories.length > 0 ? `Genre/Categories: ${book.categories.join(', ')}` :
null,
    book.publishedDate ? `Published: ${book.publishedDate}` : null,
    book.pageCount ? `Pages: ${book.pageCount}` : null,
    book.publisher ? `Publisher: ${book.publisher}` : null,
  ]
    .filter(Boolean)
    .join('\n');

  if (mode === 'improve' && book.existingDescription) {
    return `You are an expert book editor and copywriter. Improve the following book
    description.

    Book metadata:
    ${metadata}

    Current description:
    """
    ${book.existingDescription}
    """

    Requirements:
    - ${TONE_GUIDE[tone]}
    - Target length: ${LENGTH_GUIDE[length]}
    - Write in ${lang}.
    - Keep factual information accurate.
    - Improve clarity, flow, and engagement.
    - Do NOT include the book title or "by [author]" at the start – the description will
    appear alongside that info.
    - The description MUST be complete. Never stop mid-sentence. Always finish with a
    complete thought.
    - Return ONLY the improved description text, no headings or labels.`;
  }

  return `You are an expert book copywriter. Write a compelling book description.

  Book metadata:
  ${metadata}

  Requirements:
  - ${TONE_GUIDE[tone]}
  - Target length: ${LENGTH_GUIDE[length]}
  - Write in ${lang}.
  - Make it engaging and accurate for the genre.

```

- Do NOT include the book title or "by [author]" at the start – the description will appear alongside that info.
- Do NOT start with generic phrases like "This book..." or "In this book...". Start with something that hooks the reader.
- The description MUST be complete. Never stop mid-sentence. Always finish with a complete thought.
- Return ONLY the description text, no headings or labels.`;

```

}

export function generateBookDescription(params: GenerateDescriptionParams) {
  const prompt = buildPrompt(params);

  return streamText({
    model: google('gemini-2.5-flash'),
    prompt,
    maxOutputTokens: 4096,
    temperature: 0.7,
  });
}

```

```
// SQL aggregation of reading trends
```

```

export async function getReadingTrends(
  period: string,
  customStart?: string,
  customEnd?: string,
) {
  const { start, end } = dateRange(period, customStart, customEnd);

  const dailyOrders: DateCountRevenue[] = await prisma.$queryRaw`
    SELECT DATE("createdAt") as date, COUNT(*)::bigint as count,
           COALESCE(SUM("totalPrice"), 0) as revenue
    FROM "Order"
    WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
    GROUP BY DATE("createdAt")
    ORDER BY date
  `;

  const buckets = generateDailyBuckets(start, end);
  const orderMap = buildDateMap(dailyOrders);
  const salesTrend = buckets.map((d) => ({
    date: d,
    orders: Number(orderMap.get(d)?.count ?? 0),
    revenue: Number(orderMap.get(d)?.revenue ?? 0),
  }));

  const topCategories: CategoryRevenue[] = await prisma.$queryRaw`
    SELECT p."category", SUM(oi."price" * oi."quantity") as revenue,
    COUNT(*)::bigint as count
    FROM "OrderItem" oi
    JOIN "Product" p ON p."id" = oi."productId"
    JOIN "Order" o ON o."id" = oi."orderId"
    WHERE o."createdAt" >= ${start} AND o."createdAt" <= ${end}
    GROUP BY p."category"
    ORDER BY revenue DESC
    LIMIT 10
  `;

  const topBooks: BookRow[] = await prisma.$queryRaw`
    SELECT p."name", p."author", SUM(oi."quantity")::bigint as quantity,
           SUM(oi."price" * oi."quantity") as revenue
    FROM "OrderItem" oi
    JOIN "Product" p ON p."id" = oi."productId"
  `;

```

```

    JOIN "Order" o ON o."id" = oi."orderId"
    WHERE o."createdAt" >= ${start} AND o."createdAt" <= ${end}
    GROUP BY p."id", p."name", p."author"
    ORDER BY quantity DESC
    LIMIT 10
`;

const ratingDistribution: RatingRow[] = await prisma.$queryRaw`
    SELECT "rating", COUNT(*)::bigint as count
    FROM "Review"
    WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
    GROUP BY "rating"
    ORDER BY "rating"
`;

const dailyReviews: DateCount[] = await prisma.$queryRaw`
    SELECT DATE("createdAt") as date, COUNT(*)::bigint as count
    FROM "Review"
    WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
    GROUP BY DATE("createdAt")
    ORDER BY date
`;

const reviewMap = buildDateMap(dailyReviews);
const reviewsTrend = buckets.map((d) => ({
    date: d,
    reviews: Number(reviewMap.get(d)?.count ?? 0),
}));

return {
    salesTrend,
    topCategories: topCategories.map((c) => ({
        category: c.category,
        revenue: Number(c.revenue),
        count: Number(c.count),
    })),
    topBooks: topBooks.map((b) => ({
        name: b.name,
        author: b.author,
        quantity: Number(b.quantity),
        revenue: Number(b.revenue),
    })),
    ratingDistribution: ratingDistribution.map((r) => ({
        rating: r.rating,
        count: Number(r.count),
    })),
    reviewsTrend,
};
}

// Automation of analytical report generation

export async function generateAnalyticsReport(input: {
    category:
        | 'reading_trends'
        | 'community_engagement'
        | 'events_clubs'
        | 'user_behavior';
    period: string;
    customStart?: string;
    customEnd?: string;
}) {
    const session = await auth();
    if (session?.user?.role !== 'admin') {
        throw new Error('User is not authorized');
    }

```

```

}

const { start, end } = dateRange(
  input.period,
  input.customStart,
  input.customEnd,
);

const report = await prisma.analyticsReport.create({
  data: {
    category: input.category,
    title: `${input.category.replace(/_/g, ' ') } report`,
    period: input.period,
    startDate: start,
    endDate: end,
    data: {},
    status: 'processing',
    progress: 10,
    createdBy: session.user.id!,
  },
});

try {
  await prisma.analyticsReport.update({
    where: { id: report.id },
    data: { progress: 30 },
  });

  let data: unknown;

  switch (input.category) {
    case 'reading_trends':
      data = await getReadingTrends(
        input.period,
        input.customStart,
        input.customEnd,
      );
      break;
    case 'community_engagement':
      data = await getCommunityEngagement(
        input.period,
        input.customStart,
        input.customEnd,
      );
      break;
    case 'events_clubs':
      data = await getEventsClubsAnalytics(
        input.period,
        input.customStart,
        input.customEnd,
      );
      break;
    case 'user_behavior':
      data = await getUserBehavior(
        input.period,
        input.customStart,
        input.customEnd,
      );
      break;
  }

  await prisma.analyticsReport.update({
    where: { id: report.id },
    data: { progress: 90 },
  });
}

```

```

    });

    const completed = await prisma.analyticsReport.update({
      where: { id: report.id },
      data: {
        data: data as Prisma.InputJsonValue,
        status: 'completed',
        progress: 100,
      },
    });

    return { success: true, report: JSON.parse(JSON.stringify(completed)) };
  } catch (error) {
    await prisma.analyticsReport.update({
      where: { id: report.id },
      data: { status: 'failed', progress: 0 },
    });
    throw error;
  }
}

// Row types for raw SQL results
'use server';

import { auth } from '@auth';
import { prisma } from '@db/prisma';
import { Prisma } from '@src/generated/prisma';

type DateCount = { date: string; count: bigint };
type DateCountRevenue = {
  date: string;
  count: bigint;
  revenue: Prisma.Decimal;
};
type CategoryRevenue = {
  category: string;
  revenue: Prisma.Decimal;
  count: bigint;
};
type BookRow = {
  name: string;
  author: string;
  quantity: bigint;
  revenue: Prisma.Decimal;
};
type RatingRow = { rating: number; count: bigint };
type StatusCount = { status: string; count: bigint };
type FormatCount = { format: string; count: bigint };
type ClubRequestRow = { date: string; count: bigint; type: string };
type TopClubRow = { title: string; member_count: number };
type TopReviewerRow = {
  name: string;
  review_count: bigint;
  avg_rating: number;
};
type VoteRow = { is_helpful: boolean; count: bigint };
type TopBuyerRow = {
  name: string;
  order_count: bigint;
  total_spent: Prisma.Decimal;
};
type HeatmapRow = { day_of_week: number; hour: number; count: bigint };
type RoleRow = { role: string; count: bigint };

```

```

type CountOnly = { count: bigint };

function dateRange(period: string, customStart?: string, customEnd?: string) {
  const end = customEnd ? new Date(customEnd) : new Date();
  let start: Date;

  switch (period) {
    case 'last_7':
      start = new Date(end.getTime() - 7 * 86_400_000);
      break;
    case 'last_30':
      start = new Date(end.getTime() - 30 * 86_400_000);
      break;
    case 'last_90':
      start = new Date(end.getTime() - 90 * 86_400_000);
      break;
    case 'custom':
      start = customStart
        ? new Date(customStart)
        : new Date(end.getTime() - 30 * 86_400_000);
      break;
    default:
      start = new Date(end.getTime() - 30 * 86_400_000);
  }

  return { start, end };
}

function generateDailyBuckets(start: Date, end: Date) {
  const buckets: string[] = [];
  const current = new Date(start);
  while (current <= end) {
    buckets.push(current.toISOString().slice(0, 10));
    current.setDate(current.getDate() + 1);
  }
  return buckets;
}

function buildDateMap<T extends { date: string }>(rows: T[]): Map<string, T> {
  return new Map(rows.map((r) => [r.date.toString().slice(0, 10), r]));
}

export async function getReadingTrends(
  period: string,
  customStart?: string,
  customEnd?: string,
) {
  const { start, end } = dateRange(period, customStart, customEnd);

  const dailyOrders: DateCountRevenue[] = await prisma.$queryRaw`
    SELECT DATE("createdAt") as date, COUNT(*)::bigint as count,
           COALESCE(SUM("totalPrice"), 0) as revenue
    FROM "Order"
    WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
    GROUP BY DATE("createdAt")
    ORDER BY date
  `;

  const buckets = generateDailyBuckets(start, end);
  const orderMap = buildDateMap(dailyOrders);
  const salesTrend = buckets.map((d) => ({
    date: d,
    orders: Number(orderMap.get(d)?.count ?? 0),
    revenue: Number(orderMap.get(d)?.revenue ?? 0),
  }));
}

```

```

    ));

    const topCategories: CategoryRevenue[] = await prisma.$queryRaw`
      SELECT p."category", SUM(oi."price" * oi."quantity") as revenue,
      COUNT(*)::bigint as count
      FROM "OrderBook" oi
      JOIN "Product" p ON p."id" = oi."productId"
      JOIN "Order" o ON o."id" = oi."orderId"
      WHERE o."createdAt" >= ${start} AND o."createdAt" <= ${end}
      GROUP BY p."category"
      ORDER BY revenue DESC
      LIMIT 10
    `;

    const topBooks: BookRow[] = await prisma.$queryRaw`
      SELECT p."name", p."author", SUM(oi."quantity")::bigint as quantity,
      SUM(oi."price" * oi."quantity") as revenue
      FROM "OrderBook" oi
      JOIN "Product" p ON p."id" = oi."productId"
      JOIN "Order" o ON o."id" = oi."orderId"
      WHERE o."createdAt" >= ${start} AND o."createdAt" <= ${end}
      GROUP BY p."id", p."name", p."author"
      ORDER BY quantity DESC
      LIMIT 10
    `;

    const ratingDistribution: RatingRow[] = await prisma.$queryRaw`
      SELECT "rating", COUNT(*)::bigint as count
      FROM "Review"
      WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
      GROUP BY "rating"
      ORDER BY "rating"
    `;

    const dailyReviews: DateCount[] = await prisma.$queryRaw`
      SELECT DATE("createdAt") as date, COUNT(*)::bigint as count
      FROM "Review"
      WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
      GROUP BY DATE("createdAt")
      ORDER BY date
    `;

    const reviewMap = buildDateMap(dailyReviews);
    const reviewsTrend = buckets.map((d) => ({
      date: d,
      reviews: Number(reviewMap.get(d)?.count ?? 0),
    }));

    return {
      salesTrend,
      topCategories: topCategories.map((c) => ({
        category: c.category,
        revenue: Number(c.revenue),
        count: Number(c.count),
      })),
      topBooks: topBooks.map((b) => ({
        name: b.name,
        author: b.author,
        quantity: Number(b.quantity),
        revenue: Number(b.revenue),
      })),
      ratingDistribution: ratingDistribution.map((r) => ({
        rating: r.rating,
        count: Number(r.count),
      })),
    };

```

```

    reviewsTrend,
  };
}

export async function getCommunityEngagement(
  period: string,
  customStart?: string,
  customEnd?: string,
) {
  const { start, end } = dateRange(period, customStart, customEnd);

  const dailyReviews: DateCount[] = await prisma.$queryRaw`
    SELECT DATE("createdAt") as date, COUNT(*)::bigint as count
    FROM "Review"
    WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
    GROUP BY DATE("createdAt")
    ORDER BY date
  `;

  const dailyFollows: DateCount[] = await prisma.$queryRaw`
    SELECT DATE("createdAt") as date, COUNT(*)::bigint as count
    FROM "UserFollow"
    WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
    GROUP BY DATE("createdAt")
    ORDER BY date
  `;

  const followMap = buildDateMap(dailyFollows);
  const followsTrend = buckets.map((d) => ({
    date: d,
    follows: Number(followMap.get(d)?.count ?? 0),
  }));

  (r) => ({
    name: r.name,
    reviewCount: Number(r.review_count),
    avgRating: Math.round(r.avg_rating * 100) / 100,
  })),
  voteStats: voteStats.map((v) => ({
    label: v.is_helpful ? 'Helpful' : 'Not Helpful',
    count: Number(v.count),
  })),
  reportCount,
  submissionsByStatus: submissionsByStatus.map((s) => ({
    status: s.status,
    count: Number(s.count),
  })),
  }));
};
}

export async function getEventsClubsAnalytics(
  period: string,
  customStart?: string,
  customEnd?: string,
) {
  const { start, end } = dateRange(period, customStart, customEnd);

  const clubRequests: ClubRequestRow[] = await prisma.$queryRaw`
    SELECT DATE("createdAt") as date, COUNT(*)::bigint as count, "type"::text
    FROM "ClubRequest"
    WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
    GROUP BY DATE("createdAt"), "type"
    ORDER BY date
  `;
};

```

```

const activeClubs = await prisma.readingClub.count({
  where: { isActive: true },
});
const activeEvents = await prisma.event.count({ where: { isActive: true } });

const formatDistribution: FormatCount[] = await prisma.$queryRaw`
  SELECT "format"::text, COUNT(*)::bigint as count
  FROM "ClubRequest"
  WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
  GROUP BY "format"
`;

const attendanceStats: StatusCount[] = await prisma.$queryRaw`
  SELECT "status"::text, COUNT(*)::bigint as count
  FROM "Attendance"
  WHERE "createdAt" >= ${start} AND "createdAt" <= ${end}
  GROUP BY "status"
`;

const topClubs: TopClubRow[] = await prisma.$queryRaw`
  SELECT "title", array_length("memberIds", 1) as member_count
  FROM "ReadingClub"
  WHERE "isActive" = true
  ORDER BY member_count DESC NULLS LAST
  LIMIT 10
`;

return {
  clubRequests: clubRequests.map((r) => ({
    date: r.date.toString().slice(0, 10),
    count: Number(r.count),
    type: r.type,
  })),
  requestsByStatus: requestsByStatus.map((s) => ({
    status: s.status,
    count: Number(s.count),
  })),
};

```